

HTML Tag Reference

A containment tree of every HTML element — what nests inside what — with each element’s full description, placement, attributes, and values.

139 ELEMENTS & ATTRIBUTES · BUILT FOR READING & PRINTING

How to read this

Start with the **Document Tree**: it shows the nesting of every element (so you can see, for example, that `<link>` belongs in `<head>`, not `<body>`). Every name in the tree links to its full entry.

Each entry then gives: a plain-language description, a **Goes inside / Can contain** placement box, a usage example, and every element-specific attribute with its values (attribute in teal, value in amber).

The attributes shared by *all* elements (`id`, `class`, `style`, `data-*`, event handlers...) are written out in full once, in **Global Attributes**; every entry just lists their names.

At the end, an **Attribute Index** lists every attribute alphabetically with the elements it belongs to — the reverse lookup, for when you start from an attribute instead of a tag.

To print or save as PDF use `Ctrl/Cmd + P` — each section starts on a new page.

Document Tree

What nests inside what. Every element name is a link to its full entry below. Attributes shared by all elements are in the Global Attributes section.

```

├─ <!DOCTYPE html>    document type declaration — keeps the browser in standards mode
├─ <html>
│   ├── <head>    metadata, not rendered
│   │   ├── <title> <base> <meta> <link> <style>
│   │   └─ <script> <noscript> <template>
│   └─ <body>    visible content (flow content)
│       ├── Sectioning & headings
│       │   ├── <header> <nav> <main> <section> <article> <aside> <footer>
│       │   └─ <h1>...<h6> <hgroup> <address> <search>
│       ├── Grouping
│       │   ├── <p> <hr> <pre> <blockquote> <div>
│       │   └─ <figure>
│       │       └─ <figcaption>
│       ├── Lists
│       │   ├── <ul>
│       │   │   └─ <li>
│       │   ├── <ol>
│       │   │   └─ <li>
│       │   ├── <menu>
│       │   │   └─ <li>
│       │   └─ <dl>
│       │       └─ <dt> <dd>
│       ├── Tables
│       │   └─ <table>
│       │       ├── <caption>
│       │       ├── <colgroup>
│       │       └─ <col>

```

- └─ <thead>
 - └─ <tr>
- └─ <tbody>
 - └─ <tr>
- └─ <tfoot>
 - └─ <tr>
- └─ <tr>
 - └─ <th> <td>
- └─ Forms
 - └─ <form>
 - └─ <fieldset>
 - └─ <legend>
 - └─ <label> <input> <button> <textarea> <output> <progress> <meter>
 - └─ <select>
 - └─ <optgroup>
 - └─ <option>
 - └─ <datalist>
 - └─ <option>
- └─ Embedded & media
 - └─ <iframe> <canvas> <svg> <embed>
 - └─ <picture>
 - └─ <source>
 - └─ <audio>
 - └─ <source> <track>
 - └─ <video>
 - └─ <source> <track>
 - └─ <object>
 - └─ <param>
 - └─ <map>
 - └─ <area>
- └─ Inline text
 - └─ <a> <i> <u> <s> <mark> <small>
 - └─ <abbr> <cite> <q> <code> <kbd> <samp> <var> <data> <time>
 - └─ <sub> <sup> <ins> <dfn> <bdi> <bdo> <wbr>

 - └─ <ruby>
 - └─ <rt> <rp>
- └─ Interactive
 - └─ <details>
 - └─ <summary>
 - └─ <dialog>

Document Structure @ Root

The skeleton every page is built on. `<!DOCTYPE>` comes first, then a single `<html>` wrapping a `<head>` and a `<body>`.

<!DOCTYPE>

GOES INSIDE The very first line of the document, before `<html>`.

CAN CONTAIN Nothing — it is a declaration, not an element.

The document type declaration. It is NOT an HTML tag; it is an instruction to the browser telling it which version of HTML the page uses. It must be the very FIRST thing in the document, before `<html>`. In modern HTML, its only job is to keep the browser in "standards mode" (correct, spec-compliant rendering) rather than "quirks mode" (legacy bug-compatible rendering). It is case-insensitive and has no closing tag.

```
<!DOCTYPE html>
```

The modern HTML5 doctype takes no attributes. Just write `<!DOCTYPE html>`.

Legacy doctypes you may still encounter (do not use for new pages):

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN"
    "http://www.w3.org/TR/html4/strict.dtd"> - HTML 4.01 Strict
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd"> - XHTML 1.0
```

+ **global attributes** `id`, `class`, `style`, `title`, `lang`, `dir`, `hidden`, `tabindex`, `accesskey`, `contenteditable`, `draggable`, `spellcheck`, `translate`, `inputmode`, `enterkeyhint`, `inert`, `popover`, `data-*`, and `on*` event handlers — full list below.

<html>

GOES INSIDE The document root; only `<!DOCTYPE>` may precede it.

CAN CONTAIN Exactly one `<head>` followed by one `<body>`.

The root element of the page. Everything except the `<!DOCTYPE>` lives inside it. There must be exactly one per document, and it should contain exactly one `<head>` followed by one `<body>`. Setting the `lang` attribute here is strongly recommended for accessibility and SEO. This element accepts global attributes; the key ones are below.

```
<html lang="en">
  <head></head>
  <body></body>
</html>
```

lang — The primary language of the entire document. Inherited by all descendants unless they override it. (See `lang` in `00-global-attributes.html`.)

- `lang="en"` — English.
- `lang="en-GB"` — British English.
- `lang="de"` — German.

xmlns — The XML namespace. Only required when serving the document as XHTML (`application/xhtml+xml`). Ignored for normal `text/html` pages.

`xmlns="http://www.w3.org/1999/xhtml"` — The XHTML namespace.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<head>

GOES INSIDE Directly inside `<html>`, as its first child.

CAN CONTAIN Metadata: `<title>`, `<base>`, `<meta>`, `<link>`, `<style>`, `<script>`, `<noscript>`, `<template>`.

A container for metadata: information ABOUT the document that is not displayed in the page content itself. Holds the `<title>`, `<meta>` tags, `<link>` tags (stylesheets, icons), `<style>`, and `<script>` elements. Must be the first child of `<html>`. This element has no element-specific attributes; it accepts global ones.

```
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>Page Title</title>
  <link rel="stylesheet" href="styles.css" />
</head>
```

No element-specific attributes. See 11-meta-info.html for the elements that go INSIDE `<head>` (`<meta>`, `<base>`, etc.).

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<body>

GOES INSIDE Directly inside `<html>`, after `<head>`.

CAN CONTAIN Flow content (most visible page elements).

Contains all the content that is actually rendered on the page: text, images, links, tables, etc. There must be exactly one `<body>`, and it must follow `<head>`. Window-level event handlers (`onload`, `onresize`, `onunload`) are conventionally placed here. This element accepts global attributes plus several window event handlers.

```
<body>
  <h1>Welcome</h1>
  <p>This is the page content.</p>
</body>
```

onload — Runs JavaScript after the page and all its resources finish loading.
`onload="init()"` — Calls `init()` once everything is ready.

onunload / onbeforeunload — Run when the user is leaving the page.
`onbeforeunload="return confirm()"` — Can prompt before navigating away.

Deprecated presentational attributes (do NOT use; use CSS instead):

bgscolor, background, text, link, vlink, alink — colors and background.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

⌞!— ... ⌞→

GOES INSIDE Almost anywhere in the markup.

CAN CONTAIN Plain text only; cannot be nested.

An HTML comment. Anything between ⌞!— and ⌞→ is ignored by the browser and not rendered. Use comments to annotate your markup, leave notes for other developers, or temporarily disable a block of HTML. **IMPORTANT:** comments are visible to anyone who views the page source, so never put secrets in them. Comments cannot be nested, and the text inside must not contain the sequence "--".

```
⌞!— This is a comment. It will not appear on the page. ⌞→
```

```
⌞!—  
    Comments can  
    span multiple lines.  
⌞→
```

```
⌞!— <p>This paragraph is temporarily disabled.</p> ⌞→
```

A comment has no tags, attributes, or values; it is just delimited text.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and ⌞on* event handlers — full list below.

The Document Head — Metadata

These live inside ⌞head>. They describe the page and load resources, but are not rendered in the page body.

<title>

GOES INSIDE Inside ⌞head> (exactly one per document).

CAN CONTAIN Text only.

Defines the title of the document. It is shown in the browser tab, used as the default name when bookmarking, and is one of the most important pieces of text for SEO (it becomes the clickable headline in search results). There must be exactly one ⌞title>, inside ⌞head>. It may contain only text. This element has no element-specific attributes.

```
<title>My Website - Home</title>
```

No element-specific attributes. The content should be concise and descriptive; aim for roughly 50-60 characters for search-result display.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and ⌞on* event handlers — full list below.

<base>

GOES INSIDE Inside `<head>`, before any element that uses a URL (one per document).

CAN CONTAIN Empty — it is a void element with no content.

Specifies a BASE URL and/or a default target for ALL relative URLs in the document. There can be at most ONE `<base>` per page, and it must appear in `<head>` before any element that uses a URL. It is a void element. Use with care: it affects every relative link, image, and form on the page.

```
<base href="https://example.com/app/" target="_blank" />
```

href — The base URL against which all relative URLs in the document resolve.

`href="https://example.com/docs/"` — A relative `href="page.html"` now resolves to `https://example.com/docs/page.html`.

target — The default target for all hyperlinks and forms that do not specify their own target.

`target="_self"` — (default) Same browsing context.

`target="_blank"` — Open links in a new tab/window by default.

`target="_parent"` — The parent browsing context.

`target="_top"` — The full window.

+ **global attributes** `id`, `class`, `style`, `title`, `lang`, `dir`, `hidden`, `tabindex`, `accesskey`, `contenteditable`, `draggable`, `spellcheck`, `translate`, `inputmode`, `enterkeyhint`, `inert`, `popover`, `data-*`, and `on*` event handlers — full list below.

<meta>

GOES INSIDE Inside `<head>` (some, like `itemprop`, may appear in `<body>`).

CAN CONTAIN Empty — it is a void element with no content.

Represents a piece of METADATA that cannot be expressed by other head elements. It is a void element, and what it does depends on which of its attributes you use: `charset`, `name+content`, `http-equiv+content`, or `property+content` (for social/Open Graph data).

```
<meta charset="UTF-8" />
<meta name="description" content="A concise summary of the page for search engines." />
<meta name="viewport" content="width=device-width, initial-scale=1" />
<meta property="og:title" content="Shareable Title" />
```

charset — Declares the document's character encoding. Should be the FIRST thing in `<head>`. Always use UTF-8 for modern pages.

`charset="UTF-8"` — Unicode (UTF-8) encoding.

name — Names a document-level metadata property; its value goes in "content".

`name="description"` — A summary used by search engines (in results snippets).

`name="keywords"` — A comma-separated keyword list (largely ignored by search now).

`name="author"` — The page author's name.

`name="viewport"` — Controls layout on mobile (see content examples below).

`name="robots"` — Instructions for search-engine crawlers.

`name="theme-color"` — Suggests a browser UI color (mobile address bar).

`name="referrer"` — Sets the default referrer policy for the page.

`name="generator"` — The software that generated the page.

content – The VALUE associated with the name, http-equiv, or property attribute.

Its meaning depends on which it pairs with.

content="width=device-width, initial-scale=1" – (with name="viewport")

Match the device width and start at 100% zoom. Essential for responsive design.

content="noindex, nofollow" – (with name="robots") Do not index this page and do not follow its links.

content="A short page summary." – (with name="description") The summary text.

content="#0b5fff" – (with name="theme-color") A theme color.

http-equiv – Provides a pragma directive equivalent to certain HTTP response headers; the directive's value goes in "content".

http-equiv="refresh" – Reload/redirect after a delay (content="5" or content="5; url=https://example.com").

http-equiv="content-security-policy" – Declare a CSP for the page.

http-equiv="x-ua-compatible" – Legacy Internet Explorer rendering mode.

property – (not in the base HTML spec, but the de-facto standard for Open Graph and similar protocols) Names a property; value goes in "content".

property="og:title" – Title used when shared on social platforms.

property="og:description" – Description for social shares.

property="og:image" – Preview image URL for social shares.

property="og:url" – Canonical URL for the shared object.

property="og:type" – The object type (e.g. "website", "article").

media – (used with name="theme-color") Apply the value only when a media query matches, e.g. for light vs dark mode.

media="(prefers-color-scheme: dark)" – Theme color for dark mode.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and **on*** event handlers — full list below.

<link>

GOES INSIDE Inside **<head>**; body-OK rel types may also appear in **<body>**.

CAN CONTAIN Empty — it is a void element with no content.

Defines a relationship between the current document and an EXTERNAL resource. It lives in **<head>** and is a void element. Its most common job is loading a stylesheet, but it also declares icons, preloads/prefetches resources, sets canonical URLs, and more. The rel attribute determines its purpose.

```
<link rel="stylesheet" href="styles.css" />
<link rel="icon" href="/favicon.ico" sizes="any" />
<link rel="canonical" href="https://example.com/page" />
```

rel – REQUIRED. The relationship/type, which defines what the link does.

- rel="stylesheet"** – Load an external CSS file.
- rel="icon"** – The page's favicon / tab icon.
- rel="apple-touch-icon"** – Home-screen icon for iOS devices.
- rel="canonical"** – The preferred (canonical) URL for duplicate content.
- rel="alternate"** – An alternate version (e.g. an RSS feed or another language).
- rel="preload"** – Fetch a critical resource early (requires the "as" attribute).
- rel="prefetch"** – Fetch a resource likely needed for a FUTURE navigation.
- rel="preconnect"** – Open an early connection to another origin.
- rel="dns-prefetch"** – Resolve another origin's DNS early.
- rel="modulepreload"** – Preload a JavaScript module.
- rel="manifest"** – Link the web app manifest (PWA).
- rel="author"** – Link to the author/about page.
- rel="license"** – Link to the content's license.
- rel="next" / rel="prev"** – Sequence relationships for paginated content.
- rel="search"** – Link to an OpenSearch description document.

href – The URL of the linked resource.

- href="/css/main.css"** – A stylesheet on this site.
- href="https://cdn.example.com/lib.css"** – A cross-origin resource.

type – The MIME type of the linked resource.

- type="text/css"** – A CSS stylesheet.
- type="image/x-icon"** – An icon file.
- type="application/rss+xml"** – An RSS feed (with **rel="alternate"**).

media – A media query controlling WHEN the resource applies (mainly for stylesheets and icons).

- media="screen"** – Apply on screens.
- media="print"** – Apply only when printing.
- media="(max-width: 600px)"** – Apply only on small viewports.

sizes – For icons, the pixel sizes the icon is designed for.

- sizes="any"** – Scalable (e.g. an SVG/ICO icon).
- sizes="32x32"** – A 32x32 icon.
- sizes="180x180"** – A 180x180 touch icon.

as – REQUIRED with **rel="preload"/"prefetch"**: the type of resource, so the browser sets the right priority and request headers.

- as="style"** – Preloading a stylesheet.
- as="script"** – Preloading a script.
- as="font"** – Preloading a font (also requires **crossorigin**).
- as="image"** – Preloading an image.
- as="fetch"** – Preloading data fetched via **fetch()/XHR** (requires **crossorigin**).

crossorigin – Whether the resource is fetched with a CORS request. (See the example you started with - this is the same attribute.)

- crossorigin="anonymous"** – CORS request without credentials.
- crossorigin="use-credentials"** – CORS request WITH credentials.

integrity – A cryptographic hash (Subresource Integrity) the fetched resource must match, or the browser refuses to use it. Guards against tampered CDN files. Requires crossorigin for cross-origin URLs.
integrity="sha384-BASE64HASH..." – Expected hash of the resource.

hreflang – The language of the linked resource.
hreflang="en" – The resource is in English.
hreflang="es" – The resource is in Spanish.

referrerpolicy – Referrer policy used when fetching the resource.
referrerpolicy="no-referrer" – Send no Referer header.
referrerpolicy="origin" – Send only the origin.

disabled – Boolean. For rel="stylesheet", load it in a disabled state (can be toggled on later via JavaScript).
disabled – The stylesheet is initially inactive.

fetchpriority – Hints the relative priority of the fetch.
fetchpriority="high" / "low" / "auto" – Loading-priority hint.

imagesrcset / imagesizes – With rel="preload" as="image", preload a RESPONSIVE image by mirroring an 's srcset/sizes.
imagesrcset="hero-480.jpg 480w, hero-960.jpg 960w" – Candidate sources.
imagesizes="100vw" – Display-size hint for selection.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and (on*) event handlers — full list below.

<style>

GOES INSIDE Usually inside <head>.
CAN CONTAIN CSS text.

Embeds CSS rules directly in the document (usually in <head>). Use it for page-specific styles or critical CSS; for site-wide styles prefer an external file via <link rel="stylesheet">. A scoped "title" can mark it as an alternate stylesheet.

```
<style>
  body { font-family: sans-serif; }
  .highlight { background: yellow; }
</style>
```

media – A media query limiting WHEN these styles apply.
media="screen" – Apply on screens.
media="print" – Apply only when printing.
media="(max-width: 600px)" – Apply only on small viewports.

type – The styling language. Optional and effectively always CSS.
type="text/css" – CSS (the only supported value; can be omitted).

title – Names a set of alternate stylesheets the user can switch between.
title="Dark theme" – Marks this as a selectable alternate style set.

nonce – A cryptographic nonce that lets this inline style pass a strict Content-Security-Policy.
nonce="r4nd0mValue" – Must match the CSP's style-src nonce.

blocking – Marks the stylesheet as render-blocking until parsed.
blocking="render" – Block rendering until this style is processed.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and **on*** event handlers — full list below.

Sectioning & Headings

Landmark regions and headings that give the page its outline. All sit in the flow content of `<body>`.

`<h1>` to `<h6>`

GOES INSIDE Flow content, and inside `<hgroup>`, `<legend>`, `<summary>`.
CAN CONTAIN Phrasing content (text and inline elements).

Section headings, ranked by importance: `<h1>` is the most important (top-level) and `<h6>` the least. They define the document's outline, which screen readers and search engines use to understand structure. Use ONE `<h1>` per page (the main title) and do not skip levels purely for visual size (control size with CSS, not by choosing a different heading level). These elements have no element-specific attributes.

```
<h1>Main Page Title</h1>
<h2>Major Section</h2>
<h3>Subsection</h3>
<h4>Sub-subsection</h4>
<h5>Minor heading</h5>
<h6>Least important heading</h6>
```

No element-specific attributes. The obsolete "align" attribute (left/center/right/justify) must not be used; style with CSS text-align instead.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and **on*** event handlers — full list below.

`<hgroup>`

GOES INSIDE Flow content.
CAN CONTAIN One heading (`<h1>`–`<h6>`) plus optional `<p>` elements.

Groups a single heading (`<h1>`–`<h6>`) together with related content such as a subtitle/tagline expressed in one or more `<p>` elements, so they are treated as one heading unit. This element has no element-specific attributes.

```
<hgroup>
  <h1>The Daily Times</h1>
  <p>All the news that fits</p>
</hgroup>
```

No element-specific attributes.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<header>

GOES INSIDE Flow content; not inside another `<header>` / `<footer>` or `<address>` .

CAN CONTAIN Flow content (no `<header>` / `<footer>` descendants).

Introductory content for its nearest sectioning ancestor (the page, an `<article>` , or a `<section>`) - typically a logo, site title, and/or primary navigation, or an article's heading and byline. A page can have several `<header>` s (one per section). It must not be nested inside another `<header>` or a `<footer>` . This element has no element-specific attributes.

```
<header>
  <h1>My Blog</h1>
  <nav><a href="/">Home</a> <a href="/about">About</a></nav>
</header>
```

No element-specific attributes.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<nav>

GOES INSIDE Flow content.

CAN CONTAIN Flow content (most visible page elements).

A semantic section containing major navigation links - a primary menu, a table of contents, pagination, breadcrumbs. It helps assistive technology identify and jump to navigation. Reserve it for MAJOR blocks of links, not every group of links on the page (a footer's minor links rarely need it). This element has no element-specific attributes.

```
<nav aria-label="Primary">
  <ul>
    <li><a href="/">Home</a></li>
    <li><a href="/about">About</a></li>
    <li><a href="/contact">Contact</a></li>
  </ul>
</nav>
```

No element-specific attributes. (Using `aria-label`, a global ARIA attribute, is recommended when a page has more than one `<nav>` so each can be told apart.)

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<main>

GOES INSIDE Flow content; one per page; not inside article/aside/header/footer/nav.

CAN CONTAIN Flow content (most visible page elements).

Wraps the DOMINANT, unique content of the document - the central topic, not repeated chrome like the site header, nav, or footer. There must be only ONE visible `<main>` per page, and it should not be nested inside `<article>`, `<aside>`, `<footer>`, `<header>`, or `<nav>`. Helps screen-reader users skip to the main content. This element has no element-specific attributes.

```
<main>
  <h1>Article Title</h1>
  <p>The primary content of the page goes here.</p>
</main>
```

No element-specific attributes.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<section>

GOES INSIDE Flow content.

CAN CONTAIN Flow content (usually led by a heading).

A standalone THEMATIC grouping of content, typically with its own heading - a chapter, a tabbed panel, a distinct part of a page. Use it when the content forms a section of a larger whole. If the content is independently distributable, use `<article>`; if it is just a styling wrapper, use `<div>`. This element has no element-specific attributes.

```
<section>
  <h2>Features</h2>
  <p>Details about product features.</p>
</section>
```

No element-specific attributes (almost always should contain a heading).

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<article>

GOES INSIDE Flow content.

CAN CONTAIN Flow content (most visible page elements).

A self-contained, independently distributable composition - a blog post, a news story, a forum post, a product card, a comment. A good test: would it still make sense if syndicated on its own (e.g. in an RSS feed)? Articles can be nested (e.g. comments inside a post). This element has no element-specific attributes.

```
<article>
  <h2>Breaking News</h2>
  <p>The full story...</p>
</article>
```

No element-specific attributes.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<aside>

GOES INSIDE Flow content.

CAN CONTAIN Flow content (most visible page elements).

Content TANGENTIALLY related to the surrounding content - sidebars, pull quotes, related-links boxes, advertising. It is set apart from the main flow; removing it should not break the primary content's meaning. This element has no element-specific attributes.

```
<aside>
  <h3>Related</h3>
  <ul><li><a href="#">Another article</a></li></ul>
</aside>
```

No element-specific attributes.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<footer>

GOES INSIDE Flow content; not inside another `<footer>` / `<header>` or `<address>`.

CAN CONTAIN Flow content (no `<header>` / `<footer>` descendants).

Closing content for its nearest sectioning ancestor - author info, copyright, related links, contact details. Like `<header>`, a page may have multiple `<footer>`s (e.g. one per article plus one for the page). It must not be nested inside a `<header>` or another `<footer>`. This element has no element-specific attributes.

```
<footer>
  <p>&copy; 2026 Example Inc.</p>
  <nav><a href="/privacy">Privacy</a> <a href="/terms">Terms</a></nav>
</footer>
```

No element-specific attributes.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<address>

GOES INSIDE Flow content; not nested in another `<address>` .

CAN CONTAIN Flow content (no headings, sectioning, `<header>` / `<footer>`).

Supplies contact information for its nearest `<article>` or `<body>` ancestor - for example the author of an article or the owner of a site. Browsers typically render it in italics. Do NOT use it for arbitrary postal addresses (e.g. an address in a list of locations); it is specifically for contact info of the relevant person/organization. This element has no element-specific attributes.

```
<address>
  Written by <a href="mailto:jane@example.com">Jane Doe</a>.<br />
  Visit us at example.com.
</address>
```

No element-specific attributes.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<search>

GOES INSIDE Flow content.

CAN CONTAIN Flow content (the search/filter controls).

A semantic container for a set of form controls/content used to perform a SEARCH or filtering operation. It identifies the search region of a page for assistive technology (similar to `role="search"`). This element has no element-specific attributes.

```
<search>
  <form action="/results">
    <label>Search <input type="search" name="q" /></label>
    <button type="submit">Go</button>
  </form>
</search>
```

No element-specific attributes.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

Grouping Content

Block-level containers for paragraphs and related content.

<p>

GOES INSIDE Flow content.

CAN CONTAIN Phrasing content only — no block-level children.

A paragraph: a block of related running text. Browsers add vertical margin above and below by default. A `<p>` cannot contain block-level elements like `<div>`, `<ul>`, or another `<p>` (the browser will auto-close it if you try). This element has no element-specific attributes.

```
<p>This is a paragraph of text. It can be as long as you like and will wrap naturally at the edge of its container.</p>
```

No element-specific attributes. The obsolete "align" attribute must not be used; use CSS text-align instead.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<hr>

GOES INSIDE Flow content.

CAN CONTAIN Empty — it is a void element with no content.

A thematic break: represents a shift in topic within a section, such as a scene change in a story or a transition to a new subject. Browsers render it as a horizontal rule by default, but its meaning is semantic, not purely decorative. It is a void element. This element has no element-specific attributes.

```
<p>End of one topic.</p>
<hr />
<p>Start of a new topic.</p>
```

No element-specific attributes.

Obsolete presentational attributes (do NOT use; use CSS instead):

align — horizontal alignment.

size — thickness of the line.

width — length of the line.

noshade — removes the 3D shading.

color — the line color.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<pre>

GOES INSIDE Flow content.

CAN CONTAIN Phrasing content (often wraps a `<code>`).

PREformatted text. Whitespace, line breaks, and indentation inside it are preserved exactly as written, and it is rendered in a monospace font. Ideal for code listings, ASCII art, and fixed-format text. Commonly wraps a `<code>`. This element has no element-specific attributes.

```
<pre>
function greet() {
  console.log("Hello");
}
</pre>
```

No element-specific attributes. The obsolete "width" attribute (a column count) must not be used.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<blockquote>

GOES INSIDE Flow content.

CAN CONTAIN Flow content (most visible page elements).

A block-level quotation pulled from another source. Browsers indent it by default. Use the cite attribute to point to the source URL. For short, inline quotations use `<q>` instead.

```
<blockquote cite="https://example.com/speech">
  <p>The only thing we have to fear is fear itself.</p>
</blockquote>
```

cite — A URL identifying the source document or message the quote came from.
It is metadata only - browsers do not display it (you can surface it yourself, e.g. with a `<cite>` element nearby).
cite="https://example.com/article" — URL of the quoted source.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<div>

GOES INSIDE Any element that accepts flow content (or as a child of `<dl>`).

CAN CONTAIN Flow content (most visible page elements).

A generic BLOCK-LEVEL container with NO semantic meaning. Use it purely for grouping content for styling (CSS) or scripting when no more meaningful element (`<section>`, `<article>`, `<nav>`, etc.) fits. Reach for a semantic element first; use `<div>` only when none applies. This element has no element-specific attributes.

```
<div class="card">
  <h2>Title</h2>
  <p>Some content grouped for layout.</p>
</div>
```

No element-specific attributes (style and group it via class/id + CSS). Obsolete attribute (do NOT use): `align` -> CSS.

+ **global attributes** `id`, `class`, `style`, `title`, `lang`, `dir`, `hidden`, `tabindex`, `accesskey`, `contenteditable`, `draggable`, `spellcheck`, `translate`, `inputmode`, `enterkeyhint`, `inert`, `popover`, `data-*`, and `on*` event handlers — full list below.

<figure>

GOES INSIDE Flow content.

CAN CONTAIN Flow content plus an optional `<figcaption>` (first or last child).

A self-contained unit of content - an image, diagram, code listing, chart, etc. - that is referenced from the main flow and could be moved elsewhere (an appendix, a sidebar) without breaking meaning. Often paired with a `<figcaption>`. This element has no element-specific attributes.

```
<figure>
  
  <figcaption>Figure 1: Quarterly revenue for 2026.</figcaption>
</figure>
```

No element-specific attributes.

+ **global attributes** `id`, `class`, `style`, `title`, `lang`, `dir`, `hidden`, `tabindex`, `accesskey`, `contenteditable`, `draggable`, `spellcheck`, `translate`, `inputmode`, `enterkeyhint`, `inert`, `popover`, `data-*`, and `on*` event handlers — full list below.

<figcaption>

GOES INSIDE As the first or last child of `<figure>`.

CAN CONTAIN Flow content (most visible page elements).

A caption or legend for its parent `<figure>`. It must be the FIRST or LAST child of the figure. Describes the figure's content. This element has no element-specific attributes.

```
<figure>
  <figcaption>A diagram of the water cycle.</figcaption>
  
</figure>
```

No element-specific attributes.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

Lists

Ordered, unordered, and description lists, with their item elements.

GOES INSIDE Flow content.

CAN CONTAIN `<li>` elements (plus `<script>` / `<template>`).

An UNORDERED list - a collection of items whose order does NOT matter (browsers render bullets by default). Its only valid direct children are `<li>` elements (and script-supporting elements). Control the bullet style with the CSS `list-style-type` property, not HTML attributes. This element has no current element-specific attributes.

```
<ul>
  <li>Apples</li>
  <li>Bread</li>
  <li>Milk</li>
</ul>
```

No current element-specific attributes.

Obsolete attributes (do NOT use; use CSS instead):

type – bullet shape (disc/circle/square) → CSS `list-style-type`.

compact – render more compactly → CSS.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

GOES INSIDE Flow content.

CAN CONTAIN `<li>` elements (plus `<script>` / `<template>`).

An ORDERED list - items whose SEQUENCE is meaningful (steps, rankings). Browsers number the items automatically. Direct children must be `<li>`.

```
<ol start="3" reversed type="A">
  <li>Third item shown</li>
  <li>Second item shown</li>
  <li>First item shown</li>
</ol>
```

type – The numbering style for the markers.

type="1" – (default) Decimal numbers: 1, 2, 3...

type="A" – Uppercase letters: A, B, C...

type="a" – Lowercase letters: a, b, c...

type="I" – Uppercase Roman numerals: I, II, III...

type="i" – Lowercase Roman numerals: i, ii, iii...

start – The number (or letter position) the list counts up FROM.
start="5" – The first item is numbered 5.
start="3" – With **type="A"**, the first marker is "C" (the 3rd letter).

reversed – Boolean. Numbers the list in DESCENDING order (high to low).
reversed – Count down instead of up (e.g. a top-10 countdown).

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and **on*** event handlers — full list below.

<menu>

GOES INSIDE Flow content.

CAN CONTAIN `<li>` elements (plus `<script>` / `<template>`).

Semantically an unordered list of interactive items (commands/buttons) - a toolbar of actions. In practice browsers render it identically to `<ul>`, and its valid children are `<li>` elements. (Older "context menu" features using **type="context"** / **type="toolbar"** are obsolete and unsupported.) This element has no current element-specific attributes.

```
<menu>
  <li><button onclick="copy()">Copy</button></li>
  <li><button onclick="paste()">Paste</button></li>
</menu>
```

No current element-specific attributes.

Obsolete attributes (do NOT use):

type="context" / **type="toolbar"** – context-menu/toolbar modes (removed).

label – menu label (removed).

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and **on*** event handlers — full list below.

GOES INSIDE Inside `<ul>`, `<ol>`, or `<menu>`.

CAN CONTAIN Flow content (most visible page elements).

A LIST ITEM - a single entry inside a `<ul>`, `<ol>`, or `<menu>`. It may contain anything, including nested lists. Inside an `<ol>` it can override its own number with the **value** attribute.

```
<ol>
  <li>First</li>
  <li value="10">Jumps to 10</li>
  <li>Continues at 11</li>
</ol>
```

value – (only meaningful inside `<ol>`) Sets this item's number; subsequent items continue counting from it.

value="10" – This item is numbered 10.

Obsolete attribute (do NOT use):

type – per-item marker style → use CSS instead.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and **on*** event handlers — full list below.

<dl>

GOES INSIDE Flow content.

CAN CONTAIN **<dt>** and **<dd>** groups (optionally each wrapped in a **<div>**).

A **DESCRIPTION LIST**: a list of name/value groupings, such as terms and their definitions, or metadata key/value pairs. It contains **<dt>** elements (the names/terms) each followed by one or more **<dd>** elements (the descriptions). A single **<dt>** may have multiple **<dd>**, and multiple **<dt>** may share a **<dd>**. This element has no element-specific attributes.

```
<dl>
  <dt>HTML</dt>
  <dd>The markup language for structuring web pages.</dd>

  <dt>CSS</dt>
  <dd>The stylesheet language for presentation.</dd>
</dl>
```

No element-specific attributes.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and **on*** event handlers — full list below.

<dt>

GOES INSIDE Inside **<dl>** (or its **<div>**), before the **<dd>** (s).

CAN CONTAIN Flow content (no sectioning, **<header>** / **<footer>**).

A **DESCRIPTION TERM** (or name) inside a **<dl>** - the thing being described. It is followed by one or more **<dd>** elements giving its description(s). This element has no element-specific attributes.

```
<dl>
  <dt>Coffee</dt>
  <dd>A hot beverage made from roasted beans.</dd>
</dl>
```

No element-specific attributes.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and **on*** event handlers — full list below.

<dd>

GOES INSIDE Inside `<dl>` (or its `<div>`), after a `<dt>`.

CAN CONTAIN Flow content (most visible page elements).

A DESCRIPTION DETAILS element inside a `<dl>` - the description, definition, or value for the preceding `<dt>` (s). Browsers indent it by default. This element has no element-specific attributes.

```
<dl>
  <dt>Author</dt>
  <dd>Jane Doe</dd>
  <dd>jane@example.com</dd>
</dl>
```

No element-specific attributes.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

Tables

A table and the row/column-group/cell elements that must nest inside it in a fixed order.

<table>

GOES INSIDE Flow content.

CAN CONTAIN In order: optional `<caption>`, `<colgroup>` (s), `<thead>`, `<tbody>` / `<tr>` (s), `<tfoot>`.

The container for a data table. A well-formed table contains an optional `<caption>`, optional `<colgroup>`, an optional `<thead>`, one or more `<tbody>` groups, and an optional `<tfoot>`. The actual cells live in `<tr>` rows made of `<th>` (header) and `<td>` (data) cells. This element has no current element-specific attributes.

```
<table>
  <caption>Monthly sales</caption>
  <thead>
    <tr><th scope="col">Month</th><th scope="col">Revenue</th></tr>
  </thead>
  <tbody>
    <tr><td>January</td><td>$10,000</td></tr>
    <tr><td>February</td><td>$12,500</td></tr>
  </tbody>
  <tfoot>
    <tr><td>Total</td><td>$22,500</td></tr>
  </tfoot>
</table>
```

No current element-specific attributes.

Obsolete attributes (do NOT use; use CSS instead):

border – use CSS border + border-collapse.

cellpadding – use CSS padding on th/td.

cellspacing – use CSS border-spacing.

width / align / bgcolor / frame / rules – use CSS width/margin/background.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and **on*** event handlers — full list below.

<caption>

GOES INSIDE As the first child of **<table>**.

CAN CONTAIN Flow content (most visible page elements).

The title/description of the table. If present, it MUST be the first child of **<table>**. It is announced by screen readers, giving the table context. This element has no element-specific attributes.

```
<table>
  <caption>Quarterly results for 2026</caption>
  <!-- rows... -->
</table>
```

No element-specific attributes. Obsolete attribute (do NOT use): align (top/bottom/left/right) -> CSS caption-side.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and **on*** event handlers — full list below.

<colgroup>

GOES INSIDE Inside **<table>**, after **<caption>**, before any rows.

CAN CONTAIN **<col>** elements (or use its own span attribute).

Defines a GROUP of one or more columns for the purpose of applying shared styling/attributes (e.g. a background color for an entire column). It appears after **<caption>** and before **<thead>**. It contains **<col>** elements, or uses its own span attribute to cover columns directly.

```
<table>
  <colgroup>
    <col style="background: #eef;" />
    <col span="2" style="background: #efe;" />
  </colgroup>
  <!-- rows... -->
</table>
```

span – The number of consecutive columns the group covers (when it has no **<col>** children).

span="2" – This group spans two columns.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and **on*** event handlers — full list below.

<col>

GOES INSIDE Inside `<colgroup>`.

CAN CONTAIN Empty — it is a void element with no content.

Specifies properties for one or more individual columns within a `<colgroup>`. It is a void element. Mainly useful for setting column-wide CSS (background, width, borders); most other properties cannot be applied per-column this way.

```
<colgroup>
  <col />
  <col span="2" class="highlight-cols" />
</colgroup>
```

span — How many columns this single `<col>` applies to.
span="2" — Apply to the next two columns.

Obsolete attributes (do NOT use): align, valign, width, char, charoff -> CSS.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<thead>

GOES INSIDE Inside `<table>`, before `<tbody>` / `<tfoot>`.

CAN CONTAIN `<tr>` rows.

Groups the HEADER row(s) of a table. It helps assistive tech and, when a table spans multiple printed pages, lets the header repeat at the top of each page. It must come before any `<tbody>` / `<tfoot>`. This element has no element-specific attributes.

```
<thead>
  <tr><th scope="col">Name</th><th scope="col">Score</th></tr>
</thead>
```

No element-specific attributes. Obsolete attributes (do NOT use): align, valign, char, charoff -> CSS.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<tbody>

GOES INSIDE Inside `<table>`.

CAN CONTAIN `<tr>` rows.

Groups the BODY (data) rows of a table. A table may contain several `<tbody>` elements to group related rows into sections. This element has no element-specific attributes.

```
<tbody>
  <tr><td>Alice</td><td>95</td></tr>
  <tr><td>Bob</td><td>88</td></tr>
</tbody>
```

No element-specific attributes. Obsolete attributes (do NOT use): align, valign, char, charoff -> CSS.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<tfoot>

GOES INSIDE Inside `<table>`.

CAN CONTAIN `<tr>` rows.

Groups the FOOTER row(s) of a table - typically totals or summaries. Like `<thead>`, it can repeat across printed pages. This element has no element-specific attributes.

```
<tfoot>
  <tr><td>Average</td><td>91.5</td></tr>
</tfoot>
```

No element-specific attributes. Obsolete attributes (do NOT use): align, valign, char, charoff -> CSS.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<tr>

GOES INSIDE Inside `<thead>`, `<tbody>`, `<tfoot>`, or directly in `<table>`.

CAN CONTAIN `<th>` and `<td>` cells.

A TABLE ROW. It contains one or more cells - `<th>` for header cells and `<td>` for data cells. Rows live directly inside `<thead>`, `<tbody>`, or `<tfoot>` (or directly in `<table>` if those groups are omitted). This element has no element-specific attributes.

```
<tr>
  <th scope="row">Total</th>
  <td>42</td>
</tr>
```

No element-specific attributes. Obsolete attributes (do NOT use): align, valign, bgcolor, char, charoff -> CSS.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<th>

GOES INSIDE Inside `<tr>`.

CAN CONTAIN Flow content (no header/footer/sectioning).

A HEADER cell - labels a row or column of data. Browsers render it bold and centered by default. The scope attribute (and, for complex tables, headers) is important for accessibility, telling screen readers which cells a header applies to.

```
<th scope="col" abbr="Qty">Quantity</th>
```


scope – Which cells this header applies to. Crucial for accessibility.

scope="col" – Header for the column below it.

scope="row" – Header for the row beside it.

scope="colgroup" – Header for a group of columns.

scope="rowgroup" – Header for a group of rows.

colspan – How many columns this cell spans across.

colspan="2" – The cell stretches over two columns.

rowspan – How many rows this cell spans down.

rowspan="3" – The cell stretches over three rows.

headers – A space-separated list of the id values of OTHER header cells that this cell is associated with (for complex tables with layered headers).

headers="year q1" – Associated with headers id="year" and id="q1".

abbr – A short alternative label for the header, used by some assistive tech when referring to the header repeatedly.

abbr="Qty" – Short form of "Quantity".

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and **on*** event handlers — full list below.

<td>

GOES INSIDE Inside **<tr>**.

CAN CONTAIN Flow content (most visible page elements).

A DATA cell - holds the actual content of a table (numbers, text, anything). Use colspan/rowspan to merge cells, and headers to link a cell to its header cells in complex tables.

```
<td colspan="2">Merged data cell</td>
```

colspan – Number of columns this cell spans.

colspan="3" – Spans three columns.

rowspan – Number of rows this cell spans.

rowspan="2" – Spans two rows.

headers – Space-separated id list of the header cells describing this cell (for accessibility in complex tables).

headers="region q3" – Linked to those header cells.

Obsolete attributes (do NOT use): align, valign, bgcolor, width, height, nowrap, char, charoff -> use CSS.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and **on*** event handlers — full list below.

Forms & Input

A `<form>` and the controls it can contain. Some (`<select>`) have their own required children.

`<form>`

GOES INSIDE Flow content; must not be nested inside another `<form>`.

CAN CONTAIN Flow content (the controls), but no nested `<form>`.

A container that groups input controls and defines how their data is collected and sent to a server when submitted. Everything the user fills in within the form is gathered into name=value pairs and transmitted according to the method and action attributes.

```
<form action="/submit" method="post">
  <input type="text" name="username" />
  <button type="submit">Send</button>
</form>
```

action – The URL that processes the submitted form data. If omitted, the form submits to the current page URL.
action="/login" – Send data to the /login endpoint.
action="https://api.example.com/save" – Send to an absolute URL.

method – The HTTP method used to submit the form.
method="get" – (default) Appends data to the URL as a query string. Use for searches/idempotent requests; data is visible in the URL.
method="post" – Sends data in the request body. Use for changes/sensitive data; not stored in browser history or the URL.
method="dialog" – When the form is inside a `<dialog>`, submitting closes the dialog and sets its `returnValue` instead of sending data.

enctype – How the form data is encoded; only matters when `method="post"`.
enctype="application/x-www-form-urlencoded" – (default) Standard encoding.
enctype="multipart/form-data" – REQUIRED when the form contains a file input (`<input type="file">`).
enctype="text/plain" – Minimal encoding; for debugging, not real use.

name – A name for the form, allowing it to be referenced (e.g. `document.forms`).
name="checkoutForm" – Names this form.

target – Where to display the response after submitting.
target="_self" – (default) Same browsing context.
target="_blank" – A new tab/window.
target="_parent" – The parent frame.
target="_top" – The full window, breaking out of all frames.

autocomplete – Whether the browser may autofill controls in this form.
autocomplete="on" – (default) Allow autofill.
autocomplete="off" – Disable autofill for the whole form.

novalidate – Boolean. If present, the browser skips built-in validation (required, type, pattern, etc.) when the form is submitted.
novalidate – Submit without validating.

accept-charset – The character encoding(s) accepted by the server.
accept-charset="UTF-8" – Tell the server data is UTF-8 encoded.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and **on*** event handlers — full list below.

<fieldset>

GOES INSIDE Flow content (typically inside a **<form>**).

CAN CONTAIN An optional **<legend>** first, then flow content.

Groups several related controls and labels within a form, usually drawn with a border. Disabling the fieldset disables every control inside it at once.

```
<fieldset>
  <legend>Shipping address</legend>
  <label>Street <input type="text" name="street" /></label>
</fieldset>
```

disabled – Boolean. Disables ALL form controls contained in the fieldset.
disabled – Turn off the entire group.

name – A name for the group (used when scripting; not submitted as data).
name="shipping" – Names this fieldset.

form – The id of the form this fieldset is associated with (if outside it).
form="checkout" – Associate with that form.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and **on*** event handlers — full list below.

<legend>

GOES INSIDE As the first child of **<fieldset>**.

CAN CONTAIN Phrasing content and headings.

The caption/title for its parent **<fieldset>**. It must be the first child of the fieldset and describes what the grouped controls are for. This element has no element-specific attributes.

```
<fieldset>
  <legend>Payment details</legend>
  <!-- controls... -->
</fieldset>
```

No element-specific attributes.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and **on*** event handlers — full list below.

<label>

GOES INSIDE Phrasing content.

CAN CONTAIN Phrasing content including one labelable control; no nested `<label>`.

A caption for a form control. Clicking the label focuses/activates its associated control, which enlarges the clickable area and is essential for accessibility. Associate it either by the "for" attribute or by wrapping the control inside the label.

```
<label for="email">Email address</label>
<input type="email" id="email" name="email" />
```

```
<label>Subscribe <input type="checkbox" name="sub" /></label> ←!— wrapping form —>
```

for – The id of the form control this label is bound to. Must match exactly.

for="email" – Binds the label to the element with id="email".

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<input>

GOES INSIDE Phrasing content.

CAN CONTAIN Empty — it is a void element with no content.

The most versatile form control. Its TYPE attribute decides what kind of control it is - a text box, checkbox, date picker, color swatch, file chooser, and many more. It is a void element. Which other attributes apply depends on the type.

```
<input type="email" name="email" placeholder="you@example.com" required />
```

type – Selects the kind of input control. This is the most important attribute.

- `type="text"` – (default) A single-line text field.
- `type="password"` – Text field that masks the entered characters.
- `type="email"` – Text field validated as an email address.
- `type="url"` – Text field validated as a URL.
- `type="tel"` – Telephone-number field (no built-in validation; shows phone keypad on mobile).
- `type="search"` – Search field; may show a clear ("x") button.
- `type="number"` – Numeric field with spinner controls; supports min/max/step.
- `type="range"` – A slider; supports min/max/step. Value is imprecise by design.
- `type="checkbox"` – A toggle that can be checked/unchecked; use checked, value.
- `type="radio"` – One choice among a group sharing the same name attribute.
- `type="date"` – A date picker (year-month-day).
- `type="time"` – A time picker (hours, minutes).
- `type="datetime-local"` – Date and time picker with no timezone.
- `type="month"` – A year-and-month picker.
- `type="week"` – A year-and-week picker.
- `type="color"` – A color picker; value is a hex color like #ff0000.
- `type="file"` – A file chooser; pair with accept and multiple.
- `type="hidden"` – Not shown to the user; submits a fixed value.
- `type="submit"` – A button that submits the form; label set via value.
- `type="reset"` – A button that resets all controls to their defaults.
- `type="button"` – A generic push button with no default behavior (use JS).
- `type="image"` – A graphical submit button; uses src, alt, width, height.

name – The key under which this control's value is submitted. Without a name, the control's value is NOT sent. Radio buttons in one group share a name.

`name="firstName"` – Submits as firstName=<value>.

value – The control's current value. Meaning varies by type.

- `value="hello"` – Initial text for text-like inputs.
- `value="newsletter"` – The value sent when a checkbox/radio is checked.
- `value="Submit"` – The visible label for submit/reset/button types.

placeholder – Hint text shown inside an empty text-like field. It is NOT a substitute for a <label>.

`placeholder="Search products..."` – Greyed-out hint.

required – Boolean. The form cannot be submitted unless this control is filled in / checked.

`required` – Field must have a value.

disabled – Boolean. The control is greyed out, non-interactive, and its value is NOT submitted.

`disabled` – Turn the control off entirely.

readonly – Boolean. The value cannot be edited by the user but IS still submitted (unlike disabled). Not valid for checkbox/radio/file/buttons.

`readonly` – Display a fixed value the user can copy but not change.

checked – Boolean. For type="checkbox" / type="radio", makes it selected by default when the page loads.

`checked` – Pre-select this checkbox/radio.

min / max – The minimum and maximum allowed values for numeric, range, and date/time types.

`min="0"` – Smallest accepted number/date.

`max="100"` – Largest accepted number/date.

step – The granularity for numeric, range, and date/time inputs.

`step="1"` – Only whole-number increments.

`step="0.01"` – Allow two decimal places (e.g. currency).

`step="any"` – No stepping constraint; any value allowed.

minlength / maxlength – The minimum and maximum number of characters for text-like inputs.

`minlength="8"` – At least 8 characters.

`maxlength="20"` – No more than 20 characters.

pattern – A regular expression the value must match (text-like types). Pair with title to explain the rule.

`pattern="[0-9]{5}"` – Exactly five digits (e.g. a US ZIP code).

multiple – Boolean. Allows multiple values: several files (type="file") or several addresses (type="email").

`multiple` – Accept more than one value.

accept – For type="file", which file types the chooser should accept.

`accept="image/*"` – Any image type.

`accept=".pdf,.docx"` – Specific file extensions.

`accept="image/png,image/jpeg"` – Specific MIME types.

autocomplete – Hints what value the browser may autofill for this field.

`autocomplete="on" / "off"` – Enable or disable autofill.

`autocomplete="email"` – Fill with the user's saved email.

`autocomplete="current-password"` – Fill the saved password.

`autocomplete="new-password"` – Signals a password-creation field.

`autocomplete="name" / "tel" / "street-address"` – Specific data tokens.

autofocus – Boolean. The control receives keyboard focus automatically when the page loads. Use at most once per page.

`autofocus` – Focus this field on load.

list – The id of a <datalist> element providing autocomplete suggestions.

`list="browsers"` – Connects to <datalist id="browsers">.

size – The visible width of a text-like input, measured in characters (not pixels).

`size="40"` – Roughly 40 characters wide.

src / alt – For type="image" only: the image URL and its alternative text.

`src="submit.png"` – The button image.

`alt="Submit"` – Text alternative for the image button.

form – Associates the input with a <form> by id, even if it sits outside the form element in the DOM.

`form="myForm"` – Submit with <form id="myForm">.

formaction / formmethod / formenctype / formtarget / formnovalidate -

On submit-type inputs (submit/image), these OVERRIDE the parent <form>'s corresponding attributes for that one submission.

formaction="/draft" – Submit this button to a different URL.

formmethod="get" – Use GET for this button only.

formnovalidate – Skip validation when submitting via this button.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<button>

GOES INSIDE Phrasing content.

CAN CONTAIN Phrasing content, but no interactive descendants.

A clickable button. Unlike <input type="button">, its label is rich HTML content placed between the tags (so it can contain icons, formatting, etc.). ALWAYS set the type explicitly inside a form to avoid surprise submissions.

```
<button type="submit" name="action" value="save">Save changes</button>
```

type – What the button does.

type="submit" – (default inside a form!) Submits the form.

type="reset" – Resets the form's controls to their initial values.

type="button" – Does nothing on its own; behavior added via JavaScript.

name / value – Submitted as a name=value pair, but ONLY for the button that was actually clicked to submit. Lets the server tell which button was pressed.

name="action" value="delete" – Submits action=delete when clicked.

disabled – Boolean. The button is greyed out and cannot be clicked.

disabled – Disable the button.

form – The id of the form this button submits/resets, if outside it in the DOM.

form="myForm" – Associate with <form id="myForm">.

formaction / formmethod / formenctype / formtarget / formnovalidate -

On a submit button, override the form's matching attributes for this submit.

formaction="/save-draft" – Post this button to a different URL.

formnovalidate – Submit without validation (e.g. a "Save draft" button).

popovertarget / popovertargetaction – Make the button control a popover element (one with the popover global attribute).

popovertarget="menu" – Toggles the element with id="menu".

popovertargetaction="show" / "hide" / "toggle" – Which action to perform.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<select>

GOES INSIDE Phrasing content.

CAN CONTAIN `<option>` and `<optgroup>` elements.

A drop-down list of choices. It contains `<option>` elements (optionally grouped by `<optgroup>`). The selected option's value is submitted. Add the `multiple` attribute to let users pick several.

```
<select name="country">
  <option value="us">United States</option>
  <option value="ca">Canada</option>
</select>
```

name – The key under which the chosen value is submitted.

`name="country"` – Submits as `country=<selected value>`.

multiple – Boolean. Allows selecting more than one option (renders as a scrolling list box rather than a drop-down).

`multiple` – Permit multiple selections.

size – The number of options visible at once. Values > 1 show a list box.

`size="5"` – Show five rows without scrolling.

required – Boolean. A non-empty option must be chosen to submit. Pair with a placeholder option that has an empty value.

`required` – Force a selection.

disabled – Boolean. The whole control is greyed out and not submitted.

`disabled` – Disable the dropdown.

autofocus – Boolean. Focus this control on page load.

`autofocus` – Auto-focus on load.

form – The id of the form this select belongs to (if outside it in the DOM).

`form="profileForm"` – Associate with that form.

+ **global attributes** `id`, `class`, `style`, `title`, `lang`, `dir`, `hidden`, `tabindex`, `accesskey`, `contenteditable`, `draggable`, `spellcheck`, `translate`, `inputmode`, `enterkeyhint`, `inert`, `popover`, `data-*`, and `on*` event handlers — full list below.

<optgroup>

GOES INSIDE Inside `<select>`.

CAN CONTAIN `<option>` elements.

Groups related `<option>` elements inside a `<select>` under a non-selectable label, creating visual sections in the drop-down.

```
<select name="car">
  <optgroup label="German">
    <option value="bmw">BMW</option>
    <option value="vw">Volkswagen</option>
  </optgroup>
  <optgroup label="Japanese">
    <option value="honda">Honda</option>
  </optgroup>
</select>
```

label – REQUIRED. The heading text shown for the group (not selectable).
`label="Sedans"` – The group's caption.

disabled – Boolean. Greys out and disables every option within the group.
`disabled` – Disable all options in this group.

+ **global attributes** `id`, `class`, `style`, `title`, `lang`, `dir`, `hidden`, `tabindex`, `accesskey`, `contenteditable`, `draggable`, `spellcheck`, `translate`, `inputmode`, `enterkeyhint`, `inert`, `popover`, `data-*`, and `on*` event handlers — full list below.

<option>

GOES INSIDE Inside `<select>`, `<optgroup>`, or `<datalist>`.

CAN CONTAIN Text only.

A single selectable item within a `<select>`, `<optgroup>`, or `<datalist>`. The text between the tags is what the user sees; the `value` attribute is what is submitted (if omitted, the visible text is submitted instead).

```
<option value="ca" selected>Canada</option>
```

value – The data submitted when this option is chosen. Defaults to the element's text content if omitted.
`value="ca"` – Submits "ca" while the user sees the label text.

selected – Boolean. Marks this option as the default selected one on load.
`selected` – Pre-select this option.

disabled – Boolean. The option is shown but cannot be selected.
`disabled` – Make this option unselectable.

label – An alternative short label for the option (rarely needed; the text content is usually used instead).
`label="USA"` – Overrides the displayed label.

+ **global attributes** `id`, `class`, `style`, `title`, `lang`, `dir`, `hidden`, `tabindex`, `accesskey`, `contenteditable`, `draggable`, `spellcheck`, `translate`, `inputmode`, `enterkeyhint`, `inert`, `popover`, `data-*`, and `on*` event handlers — full list below.

<datalist>

GOES INSIDE Phrasing content.

CAN CONTAIN <option> elements (or phrasing-content fallback).

Provides a list of predefined <option> suggestions for an <input>. Unlike a <select>, the user can still type a free-form value; the list just offers autocomplete hints. Link it to an input via the input's "list" attribute. This element has no element-specific attributes.

```
<label>Browser:
  <input list="browsers" name="browser" />
</label>
<datalist id="browsers">
  <option value="Chrome"></option>
  <option value="Firefox"></option>
  <option value="Safari"></option>
</datalist>
```

No element-specific attributes (the connection is made via the input's "list" attribute pointing to this datalist's id).

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and <on*> event handlers — full list below.

<textarea>

GOES INSIDE Phrasing content.

CAN CONTAIN Text only (its initial value).

A multi-line plain-text input control - for comments, messages, code, etc. Unlike <input>, its initial value goes BETWEEN the tags, not in a value attribute. It resizes by default in most browsers.

```
<textarea name="comment" rows="5" cols="40" placeholder="Your message...">
Initial text goes here.</textarea>
```

name — The key under which the text is submitted.
name="bio" — Submits as bio=<text>.

rows — The visible number of text lines (height).
rows="6" — Six lines tall.

cols — The visible width in average character widths.
cols="50" — About fifty characters wide.

placeholder — Hint text shown while the field is empty.
placeholder="Type your feedback" — Greyed-out hint.

maxlength / minlength — Maximum / minimum number of characters allowed.
maxlength="500" — Cap at 500 characters.
minlength="10" — Require at least 10 characters.

wrap – How line breaks are submitted.

wrap="soft" – (default) Text wraps on screen but is submitted without automatically inserted line breaks.

wrap="hard" – Wrapped lines are submitted WITH inserted line breaks; requires the cols attribute to be set.

required – Boolean. Must be filled in before the form can submit.

required – Field is mandatory.

readonly – Boolean. Value is shown and submitted but cannot be edited.

readonly – Non-editable but submitted.

disabled – Boolean. Greyed out and not submitted.

disabled – Turn the control off.

autofocus – Boolean. Focus this control on page load.

autofocus – Auto-focus on load.

form – The id of the <form> this textarea belongs to (if outside it in the DOM).

form="contactForm" – Associate with that form.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and **on*** event handlers — full list below.

<output>

GOES INSIDE Phrasing content.

CAN CONTAIN Phrasing content (text and inline elements).

Displays the RESULT of a calculation or user action, typically updated by JavaScript. It is a semantic, accessible place to show computed output (e.g. the result of moving a slider).

```
<form oninput="result.value = Number(a.value) + Number(b.value)">
  <input type="number" id="a" value="0" /> +
  <input type="number" id="b" value="0" /> =
  <output name="result" for="a b">0</output>
</form>
```

for – A space-separated list of the ids of the elements that contributed to the result, describing the relationship for assistive tech.

for="a b" – This output is derived from elements id="a" and id="b".

name – A name for the output (so its value can be referenced via the form).

name="result" – Names this output element.

form – The id of the form this output is associated with (if outside it).

form="calc" – Associate with that form.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and **on*** event handlers — full list below.

<progress>

GOES INSIDE Phrasing content.

CAN CONTAIN Phrasing content as fallback; no nested `<progress>`.

Represents the completion progress of a TASK - a download, a form upload, an installation. If you omit the value, the bar shows an indeterminate "in progress" animation. For a static measurement within a range, use `<meter>`.

```
<label>Uploading: <progress value="32" max="100">32%</progress></label>
<progress></progress> <!-- indeterminate: still working, amount unknown -->
```

value – How much has been completed so far. If omitted, the progress bar is indeterminate. Must be between 0 and max.
value="32" – 32 units done.

max – The total amount of work that the task requires. Defaults to 1.
max="100" – Total work is 100 (so value/max is the fraction complete).

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<meter>

GOES INSIDE Phrasing content.

CAN CONTAIN Phrasing content as fallback; no nested `<meter>`.

Represents a scalar value within a KNOWN range - a gauge. Use it for things like disk usage, a score out of a maximum, or a fraction filled. Do NOT use it to indicate progress toward completion (use `<progress>` for that) or for values with no defined range.

```
<label>Disk used:
  <meter value="0.7" min="0" max="1" low="0.3" high="0.8" optimum="0.2">70%</meter>
</label>
```

value – REQUIRED. The current measured value. If outside min/max it is clamped.
value="6" – The current reading (e.g. 6 out of 10).

min – The lower bound of the range. Defaults to 0 if omitted.
min="0" – Range starts at 0.

max – The upper bound of the range. Defaults to 1 if omitted.
max="10" – Range ends at 10.

low – The value below which the measurement is considered "low" (often shown in a warning color). Must be $\geq$ min.
low="3" – Readings under 3 count as low.

high – The value above which the measurement is considered "high". Must be $\leq$ max and $\geq$ low.
high="8" – Readings over 8 count as high.

optimum – Indicates which part of the range is the ideal/optimum value. Its position relative to low/high tells the browser which zones are good.
optimum="9" – Higher values are best (the high zone is good).
optimum="1" – Lower values are best (the low zone is good).

form – Associates the meter with a `<form>` by that form's id, even if the meter is outside the form in the DOM.
form="myForm" – Links to `<form id="myForm">`.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `(on*)` event handlers — full list below.

Embedded Content & Media

Images, media players, and embedded resources — several have required child elements (`<source>`, `<track>`, `<param>`, `<area>`).

`<img>`

GOES INSIDE Phrasing content, or inside a `<picture>`.
CAN CONTAIN Empty — it is a void element with no content.

Embeds an image. It is a void element. The `src` and `alt` attributes are essential: `src` is the image source, and `alt` provides text shown if the image fails and read aloud by screen readers. For responsive images, use `srcset` + `sizes` (or wrap in `<picture>`).

```

```

src – REQUIRED. The URL of the image to display.
src="logo.png" – A relative URL.
src="https://cdn.example.com/img/hero.jpg" – An absolute URL.

alt – Alternative text describing the image. REQUIRED for accessibility.
Use an EMPTY value for purely decorative images so screen readers skip them.
alt="Bar chart of 2026 sales" – Meaningful description.
alt="" – Decorative image; intentionally ignored by assistive tech.

srcset – A set of candidate image sources for different resolutions/sizes, letting the browser pick the best one. Each entry is "URL descriptor".
srcset="img-480.jpg 480w, img-960.jpg 960w" – Width descriptors (w).
srcset="img.jpg 1x, img@2x.jpg 2x" – Pixel-density descriptors (x).

sizes – Tells the browser how wide the image will be displayed at various viewport sizes, so it can choose from `srcset` (used with w descriptors).
sizes="(max-width: 600px) 100vw, 50vw" – Full width on small screens, half width otherwise.

width / height – The intrinsic dimensions in pixels. Always set BOTH so the browser can reserve space and avoid layout shift while loading.
width="800" – 800px wide.
height="600" – 600px tall.

loading – When to load the image.
loading="eager" – (default) Load immediately.
loading="lazy" – Defer until the image nears the viewport (perf win for below-the-fold images).

decoding – Hint for how to decode the image relative to rendering.
decoding="auto" – (default) Browser decides.
decoding="async" – Decode off the main thread to avoid blocking.
decoding="sync" – Decode synchronously.

crossorigin – Whether to fetch the image with a CORS request (needed to use the image data in a <canvas> without "tainting" it).
crossorigin="anonymous" – CORS request without credentials.
crossorigin="use-credentials" – CORS request WITH credentials.

fetchpriority – Hints the relative loading priority.
fetchpriority="high" – Prioritize (e.g. the LCP hero image).
fetchpriority="low" – Deprioritize.
fetchpriority="auto" – (default) Let the browser decide.

usemap – Associates the image with a client-side image <map> by its name, prefixed with "#".
usemap="#planetmap" – Uses <map name="planetmap">.

ismap – Boolean. Only valid when the is inside an <a>. Click coordinates are sent to the server as ?x,y (a server-side image map).
ismap – Enable server-side image-map behavior.

referrerpolicy – How much referrer info to send when fetching the image.
referrerpolicy="no-referrer" – Send no Referer header.
referrerpolicy="origin" – Send only the origin.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and **on*** event handlers — full list below.

<picture>

GOES INSIDE Phrasing content.

CAN CONTAIN Zero or more `<source>`, then exactly one `<img>`.

A container that lets the browser choose among multiple image sources based on conditions (viewport width, screen density, supported formats). It holds one or more `<source>` elements and exactly one `<img>` as the fallback / actual rendered element. Use it for art direction and modern-format delivery. This element has no element-specific attributes.

```
<picture>
  <source srcset="hero.avif" type="image/avif" />
  <source srcset="hero.webp" type="image/webp" />
  <source media="(max-width: 600px)" srcset="hero-small.jpg" />
  
</picture>
```

No element-specific attributes. The decision logic lives on the child `<source>` elements (`srcset`, `type`, `media`, `sizes` – see 06-audio-video.html for the full `<source>` attribute reference). The `<img>` MUST be present and last; it is what actually renders and provides `src/alt`.

+ **global attributes** `id`, `class`, `style`, `title`, `lang`, `dir`, `hidden`, `tabindex`, `accesskey`, `contenteditable`, `draggable`, `spellcheck`, `translate`, `inputmode`, `enterkeyhint`, `inert`, `popover`, `data-*`, and `on*` event handlers — full list below.

<source>

GOES INSIDE Inside `<picture>`, `<audio>`, or `<video>` (before the fallback `<img>`).

CAN CONTAIN Empty — it is a void element with no content.

Specifies one of several alternative media resources for a `<video>`, `<audio>`, or `<picture>` parent. The browser uses the FIRST source it can handle and ignores the rest, so order them best-to-worst. It is a void element. (The attributes differ depending on whether the parent is media or `<picture>`.)

```
<video controls>
  <source src="movie.webm" type="video/webm" />
  <source src="movie.mp4" type="video/mp4" />
</video>
```

src – (for `<audio>`/`<video>`) The URL of this media resource.
`src="movie.mp4"` – One candidate media file.

type – The MIME type (and optionally codecs) of the resource, letting the browser skip formats it cannot play without downloading them.
`type="video/mp4"` – An MP4 video.
`type="audio/ogg"` – An Ogg audio file.
`type='video/mp4; codecs="avc1.42E01E"'` – MIME type plus specific codec.

srcset – (for `<picture>`) Candidate image sources with density/width descriptors, exactly like the `<img>` `srcset`.
`srcset="hero.webp 1x, hero@2x.webp 2x"` – Resolution variants.

media – (for <picture>) A media query; this source is used only if it matches.
`media="(max-width: 600px)"` – Use this image on small screens.

sizes – (for <picture>, with width descriptors) How wide the image will render at given breakpoints, guiding selection from srcset.
`sizes="(max-width: 600px) 100vw, 50vw"` – Display-width hints.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<audio>

GOES INSIDE Phrasing content.

CAN CONTAIN Optional `<source>` s, then `<track>` s, then fallback content.

Embeds sound content - music, podcasts, sound effects. Provide the audio either via a src attribute or via multiple child `<source>` elements (so the browser can pick a format it supports). Always include controls unless you build your own player. Browsers block autoplay with sound unless muted.

```
<audio controls src="song.mp3">
  Your browser does not support the audio element.
</audio>

<audio controls>
  <source src="song.ogg" type="audio/ogg" />
  <source src="song.mp3" type="audio/mpeg" />
</audio>
```

src – The URL of the audio file. Omit it if you supply `<source>` children instead.
`src="podcast.mp3"` – The audio to play.

controls – Boolean. Shows the browser's built-in play/pause/volume UI.
`controls` – Display playback controls.

autoplay – Boolean. Begin playing as soon as enough has loaded. Most browsers only honor this when the audio is also muted.
`autoplay` – Start automatically (typically requires muted).

loop – Boolean. Restart from the beginning when playback ends.
`loop` – Play on repeat.

muted – Boolean. Start with the audio muted.
`muted` – Begin silent (often needed to allow autoplay).

preload – Hint about how much to load before playback.
`preload="none"` – Do not preload (save bandwidth).
`preload="metadata"` – Load only duration/metadata.
`preload="auto"` – The browser may preload the whole file.

crossorigin – Whether to fetch with a CORS request (needed for cross-origin audio you intend to process, e.g. via Web Audio API).
crossorigin="anonymous" – CORS without credentials.
crossorigin="use-credentials" – CORS with credentials.

controlslist – Limits which buttons appear in the default controls.
controlslist="nodownload" – Hide the download option.
controlslist="noautoplayrate" – Hide playback-speed control.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and **on*** event handlers — full list below.

<video>

GOES INSIDE Phrasing content.
CAN CONTAIN Optional **<source>** s, then **<track>** s, then fallback content.

Embeds a movie/video clip. Like **<audio>**, accept either a src or child **<source>** elements, and optionally **<track>** children for captions/subtitles. It adds visual attributes (poster, width, height, playsinline) on top of the shared playback attributes.

```
<video controls width="640" height="360" poster="thumb.jpg">
  <source src="movie.webm" type="video/webm" />
  <source src="movie.mp4" type="video/mp4" />
  <track kind="subtitles" src="subs-en.vtt" srclang="en" label="English" default />
  Your browser does not support the video element.
</video>
```

src – The URL of the video file. Omit if using **<source>** children.
src="clip.mp4" – The video to play.

controls – Boolean. Show the built-in playback UI.
controls – Display player controls.

width / height – The display size of the video frame in pixels. Set both to reserve layout space and preserve aspect ratio.
width="1280" – 1280px wide.
height="720" – 720px tall.

poster – URL of an image shown before the video plays (a thumbnail/cover).
poster="cover.jpg" – Placeholder frame before playback.

autoplay – Boolean. Start playing automatically. Browsers require muted for autoplay with sound.
autoplay – Auto-start (typically with muted).

loop – Boolean. Replay from the start when finished.
loop – Play on repeat.

muted – Boolean. Start muted (commonly combined with autoplay).
muted – Begin silent.

playsinline – Boolean. On mobile (esp. iOS), play inline instead of forcing fullscreen.
playsinline – Keep playback within the page layout.

preload – How much to buffer before play (same values as <audio>).
preload="none" / "metadata" / "auto".

crossorigin – CORS mode for fetching the video (needed for cross-origin frames you draw to canvas, etc.).
crossorigin="anonymous" / "use-credentials".

controlslist – Restrict default control buttons.
controlslist="nofullscreen" – Hide the fullscreen button.
controlslist="nodownload" – Hide the download option.

disablepictureinpicture – Boolean. Prevent the Picture-in-Picture feature.
disablepictureinpicture – Disable PiP.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and **on*** event handlers — full list below.

<track>

GOES INSIDE Inside `<audio>` or `<video>`.
CAN CONTAIN Empty — it is a void element with no content.

Adds a timed text track (subtitles, captions, descriptions, chapters) to a `<video>` or `<audio>`. The text lives in an external WebVTT (.vtt) file. It is a void element placed as a child of the media element.

```
<video controls src="lecture.mp4">
  <track kind="captions" src="captions-en.vtt" srclang="en" label="English" default />
  <track kind="subtitles" src="subs-es.vtt" srclang="es" label="Español" />
</video>
```

src – REQUIRED. The URL of the WebVTT (.vtt) track file.
src="captions.vtt" – The timed-text file.

kind – The category of the text track.
kind="subtitles" – Translations / dialogue for viewers who don't know the language; shown over the video. (Has a srclang requirement.)
kind="captions" – Dialogue PLUS sound effects/cues, for deaf/HoH viewers.
kind="descriptions" – Text descriptions of the visuals, for the blind (intended to be spoken by assistive tech).
kind="chapters" – Chapter titles for navigating the media.
kind="metadata" – Machine-usable data not displayed to the user.

srclang – The language of the track's text, as a BCP 47 code. REQUIRED when **kind**="subtitles".
srclang="en" – English track.
srclang="ja" – Japanese track.

label – A human-readable title for the track, shown in the track-selection menu.
label="English (CC)" – The name the viewer sees in the menu.

default – Boolean. Marks this track as the one enabled by default (only one track per kind should have it).
default – Enable this track unless the user picks another.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<iframe> CURRENT

the modern way to embed a document.

GOES INSIDE Phrasing content.

CAN CONTAIN Fallback text (ignored by browsers that support it).

An "inline frame": embeds another HTML page (a video player, map, payment widget, third-party content, etc.) inside the current page as a rectangular region. Always provide a title for accessibility, and sandbox untrusted content for security.

```
<iframe src="https://example.com/widget"
  title="Example widget"
  width="600" height="400"
  loading="lazy"
  sandbox="allow-scripts"></iframe>
```

src – The URL of the page to embed.
src="https://maps.example.com/embed?..." – Embeds that page.

srcdoc – Inline HTML to render INSTEAD of fetching a src. Overrides src in supporting browsers (src acts as fallback).
srcdoc="<p>Hello from inside the iframe!</p>" – Renders this markup.

title – (global, but essential here) A short description of the embedded content for screen-reader users.
title="Sign-up form" – Describes what the iframe contains.

width / height – The dimensions of the frame in CSS pixels.
width="800" – 800px wide.
height="450" – 450px tall.

name – A name for the frame so links/forms with a matching target load into it.
name="preview" – target="preview" loads documents here.

loading – When the browser should load the iframe's content.
loading="eager" – (default) Load immediately.
loading="lazy" – Defer loading until the iframe is near the viewport, improving initial page performance.

sandbox – Applies extra restrictions to the embedded content for security.

An EMPTY value (`sandbox=""`) applies ALL restrictions; each token you add selectively RE-ENABLES one capability.

`sandbox=""` – Maximum lockdown (no scripts, no forms, unique origin, etc.).

`sandbox="allow-scripts"` – Re-allow JavaScript execution.

`sandbox="allow-forms"` – Re-allow form submission.

`sandbox="allow-same-origin"` – Treat content as its true origin (do NOT combine with allow-scripts for untrusted content).

`sandbox="allow-popups"` – Re-allow opening popups/new windows.

`sandbox="allow-top-navigation"` – Re-allow navigating the top-level page.

`sandbox="allow-modals"` – Re-allow alert()/confirm()/prompt().

`sandbox="allow-downloads"` – Re-allow file downloads.

(Multiple tokens are space-separated, e.g. `sandbox="allow-scripts allow-forms"`.)

allow – A Permissions Policy controlling which powerful features the embedded content may use.

`allow="fullscreen"` – Permit fullscreen requests.

`allow="camera; microphone"` – Permit camera and microphone access.

`allow="geolocation"` – Permit location access.

`allow="autoplay"` – Permit media autoplay.

allowfullscreen – Boolean. Lets the embedded content enter fullscreen mode (equivalent to `allow="fullscreen"`).

`allowfullscreen` – Permit fullscreen.

referrerpolicy – How much referrer information is sent when loading the src.

`referrerpolicy="no-referrer"` – Send no Referer header.

`referrerpolicy="origin"` – Send only the origin (scheme+host+port).

`referrerpolicy="strict-origin-when-cross-origin"` – (common default).

Obsolete presentational attributes (do NOT use; use CSS instead): `frameborder`, `marginwidth`, `marginheight`, `scrolling`, `align`.

+ **global attributes** `id`, `class`, `style`, `title`, `lang`, `dir`, `hidden`, `tabindex`, `accesskey`, `contenteditable`, `draggable`, `spellcheck`, `translate`, `inputmode`, `enterkeyhint`, `inert`, `popover`, `data-*`, and `on*` event handlers — full list below.

<embed>

GOES INSIDE Phrasing content.

CAN CONTAIN Empty — it is a void element with no content.

Embeds external content handled by a browser plugin or built-in viewer - PDFs, and similar. It is a void element with NO fallback content (use `<object>` if you need fallback). Simpler but less flexible than `<object>`.

```
<embed src="document.pdf" type="application/pdf" width="600" height="400" />
```

src – REQUIRED. The URL of the resource to embed.

`src="movie.swf"` – The external resource (e.g. a media file).

`src="document.pdf"` – A PDF to display inline.

type – The MIME type of the embedded resource, telling the browser which handler/viewer to use.
type="application/pdf" – A PDF document.
type="image/svg+xml" – An SVG image.
type="video/mp4" – An MP4 video.

width – The display width of the embedded region, in pixels.
width="800" – 800px wide.

height – The display height of the embedded region, in pixels.
height="600" – 600px tall.

(Also accepts the global title attribute, recommended for accessibility.)

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<object>

GOES INSIDE Phrasing content.
CAN CONTAIN Optional `<param>` elements, then fallback content.

Embeds an external resource - a PDF, image, another web page, or a plugin- handled object. More capable than `<embed>`: it can take `<param>` children to pass parameters and can hold FALLBACK content (shown between the tags if the resource fails to load).

```
<object data="report.pdf" type="application/pdf" width="600" height="400">
  <p>Your browser can't display the PDF. <a href="report.pdf">Download it</a>.</p>
</object>
```

data – The URL of the resource to embed. (For `<object>` this plays the role that `src` plays on `<embed>`.)
data="file.pdf" – A PDF to display.
data="page.html" – Another HTML document to embed.

type – The MIME type of the resource specified by data.
type="application/pdf" – A PDF document.
type="image/png" – A PNG image.
type="text/html" – An HTML page.

width – The display width in pixels.
width="640" – 640px wide.

height – The display height in pixels.
height="480" – 480px tall.

name – A name for the embedded browsing context (so it can be targeted by links/forms, similar to an `iframe` name).
name="viewer" – Names this object's context.

form – The id of a <form> this object is associated with (if outside it in the DOM).

form="myForm" – Associate with <form id="myForm">.

(Fallback content between the tags is shown only if the resource cannot load.)

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and **on*** event handlers — full list below.

<param>

GOES INSIDE Inside **<object>**, before any fallback content.

CAN CONTAIN Empty — it is a void element with no content.

Defines a name/value PARAMETER passed to its parent **<object>**. It is a void element placed as a child of **<object>**, before any fallback content. Largely OBSOLETE - it was mainly used to configure old plugins; modern **<object>** use (PDFs, images, pages) rarely needs it.

```
<object data="movie.swf">
  <param name="autoplay" value="true" />
  <param name="quality" value="high" />
</object>
```

name – The name of the parameter being passed to the object.

name="autoplay" – The parameter's name.

value – The value assigned to the named parameter.

value="true" – The parameter's value.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and **on*** event handlers — full list below.

<canvas>

GOES INSIDE Phrasing content.

CAN CONTAIN Fallback content shown if **<canvas>** is unsupported.

A blank, scriptable drawing surface. By itself it shows nothing; JavaScript uses its 2D context (or WebGL) to draw graphics, charts, games, and image manipulations pixel by pixel. Set width/height as ATTRIBUTES (not just CSS) to define the coordinate space; CSS only scales the rendered output.

```
<canvas id="myCanvas" width="400" height="300">
  Your browser does not support the canvas element.
</canvas>
```

width – The drawing-surface width in pixels (the coordinate system width).

Defaults to 300. Sets resolution, not just display size.

width="800" – An 800px-wide drawing buffer.

height – The drawing-surface height in pixels. Defaults to 150.

height="600" – A 600px-tall drawing buffer.

(Fallback content between the tags is shown only if canvas is unsupported.)

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<svg>

GOES INSIDE Phrasing content.

CAN CONTAIN SVG elements (`<path>`, `<circle>`, `<rect>`, `<text>`, ...).

A container for Scalable Vector Graphics - resolution-independent shapes, paths, and text defined mathematically. SVG has its own large set of child elements (`<path>`, `<circle>`, `<rect>`, `<text>`, etc.) and attributes; the container-level attributes most relevant to embedding it in HTML are below.

```
<svg width="100" height="100" viewBox="0 0 100 100" xmlns="http://www.w3.org/2000/svg">
  <circle cx="50" cy="50" r="40" fill="royalblue" />
</svg>
```

width / height – The rendered size of the SVG in the page (CSS pixels by default). Without these, the SVG uses its intrinsic/viewBox size.
width="200" – Render 200px wide.
height="150" – Render 150px tall.

viewBox – Defines the internal coordinate system as "min-x min-y width height". It maps the drawing's coordinates onto the rendered width/height, which is what makes SVG scale cleanly.
viewBox="0 0 100 100" – Coordinate space from (0,0) spanning 100x100.

xmlns – The SVG XML namespace. Required when the SVG is a standalone file; optional when inlined in an HTML document but commonly included.
xmlns="http://www.w3.org/2000/svg" – The SVG namespace.

preserveAspectRatio – How the viewBox is scaled/aligned within width/height when their aspect ratios differ.
preserveAspectRatio="xMidYMid meet" – (default) Scale to fit, centered.
preserveAspectRatio="none" – Stretch to fill, ignoring aspect ratio.
preserveAspectRatio="xMidYMid slice" – Scale to cover, cropping overflow.

fill / stroke – (presentation attributes, also settable via CSS) Default paint colors inherited by child shapes.
fill="currentColor" – Use the inherited CSS text color.
stroke="black" – Default outline color for children.

role / aria-label – (accessibility) Describe the graphic for assistive tech, since SVG is often decorative or informational.
role="img" aria-label="Company logo" – Expose it as a labeled image.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<map>

GOES INSIDE Phrasing content.

CAN CONTAIN <area> elements plus other phrasing content.

Defines a client-side image map: an overlay of clickable hotspot regions on an image. It contains one or more <area> elements and is linked to an via that image's usemap attribute.

```
<map name="planetmap">
  <area shape="circle" coords="120,80,40" href="sun.html" alt="Sun" />
  <area shape="rect" coords="200,50,260,110" href="moon.html" alt="Moon" />
</map>
```

name — REQUIRED. The map's name, referenced by an image's usemap="#name".

name="planetmap" — Referenced via usemap="#planetmap".

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and on* event handlers — full list below.

<area>

GOES INSIDE Inside a <map>.

CAN CONTAIN Empty — it is a void element with no content.

Defines one clickable region (hotspot) inside an image <map>. It is a void element. The shape and coords attributes describe the region's geometry; the other attributes behave like those on an <a> link.

```
<area shape="rect" coords="0,0,82,126" href="left.html" alt="Left section" />
```

shape — The geometry of the hotspot.

shape="rect" — A rectangle; coords are "x1,y1,x2,y2" (corners).

shape="circle" — A circle; coords are "x,y,radius" (center + radius).

shape="poly" — A polygon; coords are "x1,y1,x2,y2,x3,y3,..." (vertices).

shape="default" — The entire image (any area not covered by other hotspots).

coords — The coordinates defining the shape, in CSS pixels, comma-separated.

The number and meaning of values depend on shape (see above).

coords="34,44,270,350" — Corners of a rectangle.

coords="150,150,90" — Center (150,150) and radius 90 for a circle.

href — The URL the hotspot links to. If omitted, the area is a "dead" region.

href="products.html" — Navigate here when the region is clicked.

alt — Alternative text for the hotspot. REQUIRED when href is present.

alt="Go to products" — Describes the link for assistive tech.

target — Where to open the linked URL.

target="_blank" — New tab/window.

target="_self" — (default) Same context.

rel – The relationship of the link (same values as on `<a>`, e.g. `nofollow`, `noopener`).
`rel="noopener"` – Prevent the new page from accessing `window.opener`.

download – Boolean/string. Treats the target as a download rather than navigation.
`download` – Download the resource.
`download="map.png"` – Download with this suggested filename.

referrerpolicy / ping – Same behavior as on `<a>`: control the Referer header and notify ping URLs on click.

+ **global attributes** `id`, `class`, `style`, `title`, `lang`, `dir`, `hidden`, `tabindex`, `accesskey`, `contenteditable`, `draggable`, `spellcheck`, `translate`, `inputmode`, `enterkeyhint`, `inert`, `popover`, `data-*`, and `on*` event handlers — full list below.

Inline Text Semantics

Elements that mark up the meaning of a span of text. They go wherever phrasing content is allowed.

`<a>`

GOES INSIDE Phrasing or flow content (it is transparent).
CAN CONTAIN Same as its parent allows, but no interactive or `<a>` descendants.

The anchor element - creates a hyperlink to another page, a file, an email address, a phone number, or a location within the same page. The text (or image) between the tags is the clickable content. The `href` attribute sets the destination; without `href` it is just a placeholder anchor.

```
<a href="https://example.com" target="_blank" rel="noopener">Visit Example</a>
```

href – The destination of the link. This is the core attribute.
`href="page.html"` – A relative URL (another page on the site).
`href="https://example.com"` – An absolute URL.
`href="#section2"` – A fragment: jump to the element with `id="section2"`.
`href="mailto:hi@example.com"` – Open the user's email client.
`href="tel:+15551234567"` – Start a phone call on capable devices.
`href="/files/report.pdf"` – A root-relative URL.

target – Where to open the linked document.
`target="_self"` – (default) Open in the same tab/frame.
`target="_blank"` – Open in a new tab/window (pair with `rel="noopener"`).
`target="_parent"` – Open in the parent browsing context.
`target="_top"` – Open in the full window, escaping any frames.
`target="frameName"` – Open in the `<iframe>` with that name.

rel – The relationship between the current document and the link target.
Multiple values may be space-separated.
rel="noopener" – The new page cannot access window.opener (security; use with target="_blank").
rel="noreferrer" – Do not send the Referer header and also implies noopener.
rel="nofollow" – Tell search engines not to pass ranking credit / not to follow this link (used for paid or untrusted links).
rel="sponsored" – Marks a paid/advertising link.
rel="ugc" – Marks user-generated-content links (comments, forums).
rel="external" – The link goes to a different site.
rel="help" – Links to help/documentation.
rel="license" – Links to the licensing terms for the content.
rel="alternate" – An alternate version of the current document.
rel="next" / rel="prev" – The next / previous document in a sequence.
rel="bookmark" – A permalink for the nearest section/article.
rel="tag" – A tag/keyword that applies to the current document.

download – Makes the link DOWNLOAD the target rather than navigate to it.
download – Download using the resource's own filename.
download="annual-report.pdf" – Download with this suggested filename.

hreflang – The language of the linked document (a hint for users / search engines).
hreflang="en" – The destination is in English.
hreflang="fr" – The destination is in French.

type – The MIME type of the linked resource (a hint).
type="application/pdf" – The link points to a PDF.
type="text/html" – The link points to an HTML page.

referrerpolicy – How much referrer information to send when following the link.
referrerpolicy="no-referrer" – Send no Referer header.
referrerpolicy="origin" – Send only the origin (scheme+host+port).
referrerpolicy="strict-origin-when-cross-origin" – Common privacy default.

ping – A space-separated list of URLs to be notified (via a POST) when the link is followed - used for click tracking.
ping="https://analytics.example.com/track" – Pinged on click.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and **on*** event handlers — full list below.

GOES INSIDE Any element that accepts phrasing content.
CAN CONTAIN Phrasing content (text and inline elements).

A generic INLINE container with NO semantic meaning - the inline counterpart to `<div>`. Use it to wrap a portion of text/inline content for styling or scripting when no semantic inline element (`<em>`, `<strong>`, `<a>`, etc.) applies. This element has no element-specific attributes.

```
<p>The total is <span class="price">$42.00</span> including tax.</p>
```

No element-specific attributes (target it via class/id + CSS).

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

GOES INSIDE Phrasing content.

CAN CONTAIN Phrasing content (text and inline elements).

Marks text with stress EMPHASIS - the kind that changes the meaning of a sentence when spoken aloud. Browsers render it in italics. Nesting `<em>` inside `<em>` increases the level of emphasis. For purely visual italics with no emphasis, use `<i>`; for importance, use `<strong>`. This element has no element-specific attributes.

```
<p>You must call me <em>before</em> noon.</p>
```

No element-specific attributes.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

GOES INSIDE Phrasing content.

CAN CONTAIN Phrasing content (text and inline elements).

Marks text with strong IMPORTANCE, seriousness, or urgency - warnings, key points, terms that must not be missed. Browsers render it in bold. Nesting increases importance. For purely visual bold with no added importance, use `<b>`. This element has no element-specific attributes.

```
<p><strong>Warning:</strong> this action cannot be undone.</p>
```

No element-specific attributes.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

GOES INSIDE Phrasing content.

CAN CONTAIN Phrasing content (text and inline elements).

Marks text to be stylistically bold WITHOUT conveying extra importance - keywords, product names, or the lead sentence of a summary. If the text is genuinely important or urgent, use `<strong>` instead; if it is emphasized in speech, use `<em>`. `<b>` is the "no special meaning, just visually bold" option. This element has no element-specific attributes.

```
<p>The <b>Acme Widget 3000</b> ships next week.</p>
```

No element-specific attributes.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<i>

GOES INSIDE Phrasing content.

CAN CONTAIN Phrasing content (text and inline elements).

Marks text in an alternate voice or mood, set off from normal prose - technical terms, foreign-language phrases, thoughts, or ship/work names. Browsers render it in italics. It carries NO emphasis (use `<em>` for that); it is the "italic for a reason other than stress" element. This element has no element-specific attributes.

```
<p>The term <i lang="la">et cetera</i> means "and so on".</p>
```

No element-specific attributes.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<u>

GOES INSIDE Phrasing content.

CAN CONTAIN Phrasing content (text and inline elements).

Marks text with a non-textual annotation - rendered underlined by default. Its valid uses are narrow: e.g. marking a misspelled word or a proper name in some East Asian text. Avoid it for general underlining, because users associate underlines with links; use CSS text-decoration for visual effects. This element has no element-specific attributes.

```
<p>This word is <u>mispeled</u> (flagged by a spell checker).</p>
```

No element-specific attributes.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<s>

GOES INSIDE Phrasing content.

CAN CONTAIN Phrasing content (text and inline elements).

Marks text that is no longer accurate or relevant - struck through to show it is outdated, NOT to show a deletion edit. (For tracked deletions use `<del>`.) Browsers render it with a line through it. Typical use: a crossed-out old price next to a new one. This element has no element-specific attributes.

```
<p><s>Was $99.99</s> Now $59.99</p>
```

No element-specific attributes.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<mark>

GOES INSIDE Phrasing content.

CAN CONTAIN Phrasing content (text and inline elements).

Marks/highlights text for reference, as if drawn over with a highlighter - for example, search terms within a result, or a relevant passage the reader should notice. Browsers render it with a yellow background by default. This element has no element-specific attributes.

```
<p>Results for "html": learning <mark>html</mark> is the first step.</p>
```

No element-specific attributes.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and **on*** event handlers — full list below.

<small>

GOES INSIDE Phrasing content.

CAN CONTAIN Phrasing content (text and inline elements).

Represents "small print" - side comments, legal disclaimers, copyright lines, attributions. Renders one size smaller. It conveys the meaning of fine print, not just visually smaller text. This element has no element-specific attributes.

```
<footer><small>&copy; 2026 Example Inc. All rights reserved.</small></footer>
```

No element-specific attributes.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and **on*** event handlers — full list below.

<sub>

GOES INSIDE Phrasing content.

CAN CONTAIN Phrasing content (text and inline elements).

Subscript: text set slightly below the baseline in a smaller size. Use it where it carries meaning - chemical formulas, mathematical indices, footnote references - not for purely typographic effects. This element has no element-specific attributes.

```
<p>Water is H<sub>2</sub>O.</p>
```

No element-specific attributes.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and **on*** event handlers — full list below.

<sup>

GOES INSIDE Phrasing content.

CAN CONTAIN Phrasing content (text and inline elements).

Superscript: text set slightly above the baseline in a smaller size. Use it where it carries meaning - exponents, ordinal indicators, footnote markers. This element has no element-specific attributes.

```
<p>E = mc<sup>2</sup>. See note<sup>1</sup>.</p>
```

No element-specific attributes.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<abbr>

GOES INSIDE Phrasing content.

CAN CONTAIN Phrasing content (text and inline elements).

Marks an abbreviation or acronym. Pairing it with a title attribute lets browsers show the full expansion on hover and lets screen readers announce it. Use it the first time an abbreviation appears in your text. This element has no element-specific attributes; title (global) is key.

```
<p>The <abbr title="World Health Organization">WHO</abbr> issued guidance.</p>
```

title – (global attribute) The full expansion of the abbreviation. Strongly recommended so the meaning is available to users and assistive tech.
title="Cascading Style Sheets" – Expansion shown for the text "CSS".

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<cite>

GOES INSIDE Phrasing content.

CAN CONTAIN Phrasing content (text and inline elements).

Marks the TITLE of a creative work (a book, film, song, paper, etc.). Browsers render it in italics. It refers to the work's title, not to the author or to a person's name. This element has no element-specific attributes.

```
<p>My favorite novel is <cite>Pride and Prejudice</cite>.</p>
```

No element-specific attributes. (Do not confuse this with the "cite" ATTRIBUTE used on `<blockquote>`, `<q>`, `<ins>`, and `<del>`.)

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<q>

GOES INSIDE Phrasing content.

CAN CONTAIN Phrasing content (text and inline elements).

A short, **INLINE** quotation. Browsers automatically add the correct quotation marks around the content (locale-aware), so do not type your own. For longer, block-level quotations use `<blockquote>`.

```
<p>She said <q cite="https://example.com">the code works</q> and left.</p>
```

cite – A URL identifying the source of the quotation. Metadata only; not displayed by browsers.

cite="https://example.com/source" – URL the quote is taken from.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<code>

GOES INSIDE Phrasing content.

CAN CONTAIN Phrasing content (text and inline elements).

Marks a fragment of computer code inline. Browsers render it in a monospace font. For a whole multi-line code block, wrap a `<code>` inside a `<pre>` so the whitespace and line breaks are preserved. This element has no element-specific attributes.

```
<p>Use the <code>map()</code> method to transform an array.</p>
```

No element-specific attributes.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<kbd>

GOES INSIDE Phrasing content.

CAN CONTAIN Phrasing content (text and inline elements).

Marks **KEYBOARD** (or other user) input - keys the user should press or text they should type. Browsers render it in monospace. Nest `<kbd>` elements to represent key combinations. This element has no element-specific attributes.

```
<p>Save with <kbd><kbd>Ctrl</kbd>+<kbd>S</kbd></kbd>. </p>
```

No element-specific attributes.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<samp>

GOES INSIDE Phrasing content.

CAN CONTAIN Phrasing content (text and inline elements).

Marks SAMPLE OUTPUT from a computer program or system - the text a program prints. Browsers render it in monospace. The counterpart to `<kbd>` (input). This element has no element-specific attributes.

```
<p>The program printed <samp>File not found.</samp></p>
```

No element-specific attributes.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<var>

GOES INSIDE Phrasing content.

CAN CONTAIN Phrasing content (text and inline elements).

Marks a VARIABLE - a variable name in math or programming, or a placeholder. Browsers render it in italics. Pairs naturally with `<code>` and `<samp>`. This element has no element-specific attributes.

```
<p>The area is <var>w</var> &times; <var>h</var>.</p>
```

No element-specific attributes.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<data>

GOES INSIDE Phrasing content.

CAN CONTAIN Phrasing content (text and inline elements).

Links a piece of human-readable content with a MACHINE-READABLE value via the value attribute - useful for things scripts or microdata parsers need to read precisely (product IDs, numeric values). For dates/times specifically, use `<time>` instead.

```
<p>Product: <data value="SKU-00042">Deluxe Widget</data></p>
```

value — REQUIRED. The machine-readable form of the element's content.

value="SKU-00042" — The machine value behind the visible text.

value="398" — A numeric value behind a formatted label.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<time>

GOES INSIDE Phrasing content.
CAN CONTAIN Phrasing content (text and inline elements).

Marks a specific date, time, or duration in a MACHINE-READABLE format via the `datetime` attribute, while showing human-friendly text to the reader. Helps search engines and calendar tools parse the value precisely.

```
<p>The event starts <time datetime="2026-05-29T19:00">Friday at 7pm</time>.</p>
```

datetime — The machine-readable value in a valid date/time format. If the element's text is already a valid date/time, this can be omitted.

- `datetime="2026-05-29"` — A date (YYYY-MM-DD).
- `datetime="19:00"` — A time (HH:MM).
- `datetime="2026-05-29T19:00:00+02:00"` — Date and time with timezone offset.
- `datetime="2026-W22"` — A week (year + week number).
- `datetime="PT2H30M"` — A duration (here, 2 hours 30 minutes).

+ **global attributes** `id`, `class`, `style`, `title`, `lang`, `dir`, `hidden`, `tabindex`, `accesskey`, `contenteditable`, `draggable`, `spellcheck`, `translate`, `inputmode`, `enterkeyhint`, `inert`, `popover`, `data-*`, and `on*` event handlers — full list below.

GOES INSIDE Phrasing or flow content (transparent).
CAN CONTAIN Same as its parent allows (transparent).

Marks text that has been DELETED from the document, typically shown with a strikethrough. Often paired with `<ins>` to show an edit. Use it to represent tracked changes or revisions.

```
<p>Price: <del datetime="2026-01-01T09:00Z">$50</del> <ins>$40</ins></p>
```

cite — A URL pointing to a document that explains the change/edit.

- `cite="https://example.com/changelog"` — Source explaining the deletion.

datetime — The date (and optionally time) the change was made, in ISO 8601 format. Machine-readable metadata; not displayed by browsers.

- `datetime="2026-01-01"` — Date only.
- `datetime="2026-01-01T09:30:00Z"` — Date and time in UTC.

+ **global attributes** `id`, `class`, `style`, `title`, `lang`, `dir`, `hidden`, `tabindex`, `accesskey`, `contenteditable`, `draggable`, `spellcheck`, `translate`, `inputmode`, `enterkeyhint`, `inert`, `popover`, `data-*`, and `on*` event handlers — full list below.

<ins>

GOES INSIDE Phrasing or flow content (transparent).
CAN CONTAIN Same as its parent allows (transparent).

Marks text that has been INSERTED into the document, typically shown underlined. The counterpart to `<del>`. Use it to represent additions in a tracked-changes or revision context.

```
<p>The meeting is on <del>Monday</del> <ins datetime="2026-01-02">Tuesday</ins>.</p>
```

cite – A URL pointing to a document explaining why the text was inserted.
`cite="https://example.com/changelog"` – Source explaining the insertion.

datetime – The date/time of the insertion in ISO 8601 format. Metadata only.
`datetime="2026-01-02T14:00:00Z"` – When the text was added.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<dfn>

GOES INSIDE Phrasing content.

CAN CONTAIN Phrasing content, but no nested `<dfn>`.

Marks the DEFINING INSTANCE of a term - the place in the text where the term is actually being defined (not every later mention). Browsers render it in italics. If it has a title, that title is taken to be the term being defined. This element has no element-specific attributes; title (global) has special meaning.

```
<p>A <dfn title="HyperText Markup Language">HTML</dfn> document is plain text
marked up with tags.</p>
```

title – (global) When present, it should hold the exact term being defined.
`title="Application Programming Interface"` – The term defined by "API".

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<bdi>

GOES INSIDE Phrasing content.

CAN CONTAIN Phrasing content (text and inline elements).

Bi-Directional Isolation. Isolates a span of text whose direction might differ from the surrounding text (e.g. a user-supplied name in Arabic embedded in an English sentence), so it cannot reorder the text around it. Essential when inserting unknown-direction user content. This element has no element-specific attributes (its dir defaults to "auto").

```
<p>User <bdi>إيان</bdi> posted a comment.</p>
```

dir – (global) Defaults to "auto" on `<bdi>`, which is the whole point: the browser detects direction automatically and isolates it.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<bdo>

GOES INSIDE Phrasing content.
CAN CONTAIN Phrasing content (text and inline elements).

Bi-Directional Override. FORCES the text direction of its content, overriding the natural directionality of the characters. Unlike `<bdi>`, which isolates, `<bdo>` explicitly reverses. The `dir` attribute is REQUIRED.

```
<bdo dir="rtl">This text will be forced right-to-left.</bdo>
```

dir – REQUIRED. The direction to force.
dir="ltr" – Force left-to-right rendering.
dir="rtl" – Force right-to-left rendering (characters laid out R-to-L).

+ **global attributes** `id`, `class`, `style`, `title`, `lang`, `dir`, `hidden`, `tabindex`, `accesskey`, `contenteditable`, `draggable`, `spellcheck`, `translate`, `inputmode`, `enterkeyhint`, `inert`, `popover`, `data-*`, and `on*` event handlers — full list below.

<ruby> / <rt> / <rp>

GOES INSIDE `<ruby>` sits in phrasing content; `<rt>` and `<rp>` go inside `<ruby>`.
CAN CONTAIN `<ruby>`: base text with `<rt>` / `<rp>`. `<rt>`: phrasing content. `<rp>`: text only.

Ruby annotations show small pronunciation or meaning text above (or beside) base characters - widely used for East Asian typography such as furigana over kanji. `<ruby>` is the container. `<rt>` holds the annotation (the small text shown above the base). `<rp>` holds fallback parentheses shown ONLY in browsers that do not support ruby, so the annotation degrades gracefully. These elements have no element-specific attributes.

```
<ruby>
  漢 <rp>(</rp><rt>kan</rt><rp>)</rp>
  字 <rp>(</rp><rt>ji</rt><rp>)</rp>
</ruby>
```

None of `<ruby>`, `<rt>`, or `<rp>` have element-specific attributes.

+ **global attributes** `id`, `class`, `style`, `title`, `lang`, `dir`, `hidden`, `tabindex`, `accesskey`, `contenteditable`, `draggable`, `spellcheck`, `translate`, `inputmode`, `enterkeyhint`, `inert`, `popover`, `data-*`, and `on*` event handlers — full list below.

<wbr>

GOES INSIDE Phrasing content.
CAN CONTAIN Empty — it is a void element with no content.

Word Break Opportunity. A void element that marks a point where the browser MAY insert a line break if needed, inside an otherwise unbreakable string (like a long URL or identifier). It does not force a break; it only permits one. Unlike `­`; it adds no hyphen. This element has no element-specific attributes.

```
<p>https://example.com/<wbr />very<wbr />long<wbr />path<wbr />segment</p>
```

No element-specific attributes.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

GOES INSIDE Phrasing content.

CAN CONTAIN Empty — it is a void element with no content.

Inserts a single line break inside flowing text. It is a void (self-closing) element with no content. Use it **ONLY** for line breaks that are part of the content itself (e.g. lines of a poem or a postal address) - NOT to create spacing between paragraphs or blocks; use CSS margins for that. This element has no element-specific attributes.

```
<p>123 Main Street<br />
  Springfield, USA</p>
```

No element-specific attributes. The obsolete "clear" attribute (used to clear floats) must not be used; use the CSS clear property instead.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

Interactive Elements

Elements the user can open, close, or operate.

<details>

GOES INSIDE Flow content.

CAN CONTAIN A `<summary>` first, then flow content.

A disclosure widget the user can expand or collapse to reveal extra content - an accordion or "read more" / FAQ item. Its first child should be a `<summary>` providing the always-visible label; everything after it is the hideable content.

```
<details name="faq">
  <summary>What is your return policy?</summary>
  <p>You may return items within 30 days for a full refund.</p>
</details>
```

open — Boolean. If present, the details are expanded when the page loads.

open — Start in the expanded (open) state.

name — Groups multiple `<details>` into an exclusive accordion: opening one with the same name automatically closes the others.

name="faq" — All `<details name="faq">` behave as one accordion group.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<summary>

GOES INSIDE As the first child of `<details>`.

CAN CONTAIN Phrasing content, or one heading.

The visible heading/label for its parent `<details>`. Clicking it toggles the disclosure open or closed. It must be the **FIRST** child of the `<details>`. This element has no element-specific attributes.

```
<details>
  <summary>Click to show more</summary>
  <p>Hidden content revealed on click.</p>
</details>
```

No element-specific attributes.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

<dialog>

GOES INSIDE Flow content.

CAN CONTAIN Flow content (most visible page elements).

A modal or non-modal dialog box / popup window - a confirmation prompt, a lightbox, a settings panel. It is shown via JavaScript: `showModal()` opens it modally (with a backdrop and focus trapping), `show()` opens it non-modally, and `close()` dismisses it. Forms with `method="dialog"` close it on submit.

```
<dialog id="confirmBox">
  <form method="dialog">
    <p>Are you sure?</p>
    <button value="cancel">Cancel</button>
    <button value="ok">OK</button>
  </form>
</dialog>
```

open – Boolean. If present, the dialog is shown when the page loads (as a NON-modal dialog). Note: opening via JavaScript `showModal()` is usually preferred over hard-coding this attribute.

open – Display the dialog immediately (non-modal).

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

Scripting

Elements for running or holding scripts and inert template markup.

<script>

GOES INSIDE `<head>`, `<body>`, or anywhere phrasing/flow content is allowed.

CAN CONTAIN Script code, or data for non-JS types (empty when src is set).

Embeds or references executable code, normally JavaScript. With a src attribute it loads an external file; without src, the code goes between the tags. It can also hold non-executed data blocks (e.g. JSON) via the type attribute. Placement and the async/defer attributes determine WHEN it runs relative to HTML parsing.

```
<script src="app.js" defer></script>

<script>
  console.log("Inline script");
</script>
```

src – The URL of an external script file. When present, the element must be empty (no inline code between the tags).
`src="main.js"` – A relative URL.
`src="https://cdn.example.com/lib.js"` – A cross-origin script.

type – How the browser interprets the script's content.
`(omitted)` – Defaults to "text/javascript"; a classic script. No need to set.
`type="module"` – A JavaScript MODULE: supports import/export, runs in strict mode, is deferred by default, and executes only once.
`type="importmap"` – The content is a JSON import map that controls how module specifiers (bare names) resolve. Must appear before module scripts.
`type="application/json"` – Inert JSON DATA (not executed); read later by other scripts. Any non-JS type makes the block non-executable.
`type="text/javascript"` – An explicit classic script (same as omitting it).

async – Boolean. For EXTERNAL classic scripts: download in parallel with HTML parsing and execute as soon as ready (order NOT guaranteed). Best for independent scripts like analytics. Ignored for inline scripts.
`async` – Load asynchronously; run at the first opportunity.

defer – Boolean. For EXTERNAL classic scripts: download in parallel but defer EXECUTION until the HTML is fully parsed, preserving source order. Ideal for scripts that touch the DOM. Ignored for inline scripts. (Module scripts are deferred by default.)
`defer` – Execute after parsing, in order.

crossorigin – Whether to fetch the external script with a CORS request. Also enables detailed error reporting for cross-origin scripts.
`crossorigin="anonymous"` – CORS request without credentials.
`crossorigin="use-credentials"` – CORS request WITH credentials.

integrity – A Subresource Integrity hash the fetched script must match, or the browser refuses to run it (protects against tampered CDN files).
Requires crossorigin for cross-origin scripts.
integrity="sha384-BASE64HASH..." – Expected hash of the script file.

nomodule – Boolean. The script runs ONLY in browsers that do NOT support ES modules - a fallback for legacy browsers paired with a **type="module"** version for modern ones.
nomodule – Legacy-only fallback script.

referrerpolicy – How much referrer info is sent when fetching the script.
referrerpolicy="no-referrer" – Send no Referer header.
referrerpolicy="origin" – Send only the origin.
referrerpolicy="strict-origin-when-cross-origin" – Common privacy default.

nonce – A cryptographic nonce allowing this script to run under a strict Content-Security-Policy.
nonce="r4nd0mValue" – Must match the CSP's script-src nonce.

fetchpriority – Hints the relative priority of fetching the script.
fetchpriority="high" / "low" / "auto" – Loading-priority hint.

blocking – Marks the script as render-blocking until it is fetched/executed.
blocking="render" – Block rendering until this script is handled.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and **on*** event handlers — full list below.

<noscript>

GOES INSIDE In **<head>** (limited) or in **<body>** flow content.
CAN CONTAIN Fallback content for when scripting is disabled.

Provides fallback content shown ONLY when scripting is disabled or unsupported in the browser. In **<head>** it may contain **<link>**, **<style>**, and **<meta>**; in **<body>** it may contain ordinary flow content. Use it to tell users to enable JavaScript, or to supply a no-JS alternative. This element has no element-specific attributes.

```
<noscript>
  <p>This site works best with JavaScript enabled. Please turn it on.</p>
</noscript>
```

No element-specific attributes.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and **on*** event handlers — full list below.

<template>

GOES INSIDE Almost anywhere, including as a child of `<table>`, `<select>`, etc.

CAN CONTAIN Any inert markup, cloned later by script.

Holds a fragment of HTML that is NOT rendered when the page loads and is inert (its images do not load, its scripts do not run). It exists to be cloned and inserted later by JavaScript. Access its contents via the element's `.content` property. This element has no element-specific attributes (a "shadowrootmode" attribute exists for declarative shadow DOM, used in advanced component scenarios).

```
<template id="row-template">
  <tr><td class="name"></td><td class="email"></td></tr>
</template>
```

No commonly-used element-specific attributes for ordinary cloning use.

shadowrootmode – (advanced) Declares a declarative shadow root; values "open" or "closed". Only relevant when building web components.

+ **global attributes** id, class, style, title, lang, dir, hidden, tabindex, accesskey, contenteditable, draggable, spellcheck, translate, inputmode, enterkeyhint, inert, popover, data-*, and `on*` event handlers — full list below.

Global Attributes

Every element accepts these (a few have no visible effect on non-rendered elements like `<head>`). Each element entry above lists their names; here is the full detail.

id

Assigns a unique identifier to the element. Must be unique within the whole document. Used as a CSS hook (`#myid`), a JavaScript lookup target (`document.getElementById`), and as a fragment link target (`href="#myid"`).

```
<div id="main-header">...</div>
```

id – The value can be any string with no spaces. It is case-sensitive.
`id="main-header"` – A descriptive, unique name for this one element.

class

Assigns one or more class names to the element. Unlike `id`, the same class can be reused on many elements. Primary hook for CSS (`.myclass`) and for grouping elements in JavaScript (`getElementsByClassName` / `querySelectorAll`).

```
<p class="lead intro highlighted">...</p>
```

class – A space-separated list of one or more class names.
`class="btn"` – A single class.
`class="btn btn-primary large"` – Multiple classes applied at once.

style

Applies inline CSS directly to this one element. Highest specificity of the normal CSS sources, so it overrides stylesheet rules. Generally avoid for maintainability; prefer classes. Useful for one-off or dynamic styling.

```
<span style="color: red; font-weight: bold;">...</span>
```

style – A string of CSS declarations (property: value; pairs).

style="color: red;" – A single declaration.

style="margin: 0; padding: 8px; background: #eee;" – Multiple declarations.

title

Advisory information shown as a tooltip on hover. Also read by some assistive technologies. Do NOT rely on it for essential information, since it is not available on touch devices and is easy to miss.

```
<abbr title="HyperText Markup Language">HTML</abbr>
```

title – Any plain-text string. Newlines (
) are honored in the tooltip.

title="Click to expand" – A short hint.

lang

Declares the human language of the element's content. Helps screen readers choose the correct pronunciation, helps search engines, and enables language-specific CSS (:lang()) and hyphenation/spellcheck behavior.

```
<p lang="fr">Bonjour le monde</p>
```

lang – A valid BCP 47 language tag.

lang="en" – English.

lang="en-US" – English, United States variant.

lang="es" – Spanish.

lang="zh-Hans" – Chinese, Simplified script.

lang="" – Empty value means the language is unknown.

dir

Sets the text directionality of the element's content.

```
<p dir="rtl">مرحبا بالعالم</p>
```

dir – Controls the base direction of text.

dir="ltr" – Left-to-right (English, most European languages).

dir="rtl" – Right-to-left (Arabic, Hebrew).

dir="auto" – The browser decides based on the first strong directional character in the content. Useful for user-generated text.

hidden

Indicates the element is not yet, or no longer, relevant. The browser does not render hidden elements. Use it for content toggled by scripting rather than for things hidden purely for layout (use CSS for that).

```
<div hidden>This is not shown.</div>
```

```
<div hidden="until-found">Revealed by in-page find or scroll-to-text.</div>
```

hidden – A boolean-like attribute.

hidden – (or `hidden=""`) Element is fully hidden and removed from layout.

hidden="until-found" – Element is hidden but can be revealed by the browser's "find in page" feature or a scroll-to-text-fragment link; fires a "beforematch" event when revealed.

tabindex

Controls whether and in what order an element is focusable via the Tab key.

```
<div tabindex="0">Focusable div</div>
```

tabindex – An integer.

tabindex="0" – Element becomes focusable and joins the natural tab order in source order. Most common value for custom controls.

tabindex="-1" – Element is focusable by script/click but is SKIPPED by Tab navigation. Useful for focus targets like modal dialogs.

tabindex="1" – (any positive number) Element is focused before `tabindex="0"` elements, ordered by the number. Discouraged: it creates confusing, hard-to-maintain tab orders.

accesskey

Suggests a keyboard shortcut to activate or focus the element. The exact modifier keys vary by browser/OS (often Alt, or Alt+Shift, or Ctrl+Alt).

```
<button accesskey="s">Save</button>
```

accesskey – One or more single characters separated by spaces; the browser uses the first one it supports.

accesskey="s" – Activates this element via the platform's accesskey combo.

contenteditable

Makes the element's content directly editable by the user, turning it into a basic rich-text editing region.

```
<div contenteditable="true">Type here... </div>
```

contenteditable – Controls editability.

contenteditable="true" – (or `contenteditable=""`) Content is editable.

contenteditable="false" – Content is not editable.

contenteditable="plaintext-only" – Editable, but rich formatting is stripped; only plain text is kept.

draggable

Specifies whether the element can be dragged using the native HTML Drag-and-Drop API. Links and images are draggable by default.

```

```

draggable – Must be an explicit value (it is NOT a boolean attribute).

draggable="true" – Element can be dragged.

draggable="false" – Element cannot be dragged.

spellcheck

Hints whether the element should be checked for spelling/grammar errors. Mainly relevant for editable content (inputs, textareas, contenteditable).

```
<textarea spellcheck="true"></textarea>
```

spellcheck – Controls spell checking.

spellcheck="true" – (or **spellcheck=""**) Enable spell checking.

spellcheck="false" – Disable spell checking.

translate

Tells translation tools whether the element's text should be translated.

```
<span translate="no">BrandName</span>
```

translate – Controls translation.

translate="yes" – (or **translate=""**) Content should be translated.

translate="no" – Content should NOT be translated (brand names, code).

inputmode

Hints which virtual keyboard layout to show on touch devices for editable elements. Does not affect validation; purely a UX hint.

```
<input inputmode="numeric" />
```

inputmode – Suggested keyboard type.

inputmode="none" – No virtual keyboard (custom input UI).

inputmode="text" – Standard text keyboard (default).

inputmode="decimal" – Numeric keypad with a decimal separator.

inputmode="numeric" – Digits only.

inputmode="tel" – Telephone keypad.

inputmode="search" – Text keyboard with a "Search" enter key.

inputmode="email" – Text keyboard optimized for email (@ key).

inputmode="url" – Text keyboard optimized for URLs (/ and . keys).

enterkeyhint

Hints what label/icon the Enter key should show on a virtual keyboard.

```
<input enterkeyhint="search" />
```

enterkeyhint – Label for the Enter/Return action key.

enterkeyhint="enter" – Insert a new line.

enterkeyhint="done" – Nothing more to input; close the keyboard.

enterkeyhint="go" – Take the user to the target of their input.

enterkeyhint="next" – Move to the next field.

enterkeyhint="previous" – Move to the previous field.

enterkeyhint="search" – Submit a search.

enterkeyhint="send" – Send a message.

inert

Marks the element (and all its descendants) as inert: non-clickable, non-focusable, and hidden from assistive tech and find-in-page. Useful for "freezing" background content behind a modal.

```
<main inert>Background content behind an open dialog.</main>
```

inert – A boolean attribute (presence = on).

inert – The subtree cannot be interacted with or focused.

popover

Turns the element into a popover that can be shown on top of other content in the browser's top layer, controlled by a button with `popovertarget`.

```
<div id="menu" popover>Popover content</div>
<button popovertarget="menu">Toggle</button>
```

popover – Defines popover behavior.

popover="auto" – (or `popover=""`) Light-dismiss: closing on outside click or Escape, and only one auto popover open per ancestor chain.

popover="manual" – Stays open until explicitly closed; not light-dismissed.

popover="hint" – Hint popover (e.g. tooltip-like), does not close other auto popovers when shown.

data-*

Custom data attributes. Store private, page-specific data on any element without affecting layout or semantics. Read in JS via `element.dataset`. The name after "data-" becomes a camelCased dataset key.

```
<article data-author-id="42" data-published-date="2026-01-01">... </article>
```

data-* – Any attribute whose name starts with "data-". The rest of the name must be lowercase and may contain hyphens.

data-user-id="42" – Read in JS as `element.dataset.userId`.

data-state="open" – Read as `element.dataset.state`; also CSS-selectable via `[data-state="open"]`.

EVENT HANDLER ATTRIBUTES

These run JavaScript when an event fires. They are global (most apply to most elements). Inline handlers are convenient but `addEventListener()` in a separate script is generally preferred for maintainability and CSP safety. Below are the most commonly used ones grouped by category.

```
<button onClick="doSomething()">Click me</button>
```

Mouse / pointer events:

- onclick** – Element was clicked.
- ondblclick** – Element was double-clicked.
- onmousedown** – A mouse button was pressed over the element.
- onmouseup** – A mouse button was released over the element.
- onmouseover** – Pointer moved onto the element.
- onmouseout** – Pointer moved off the element.
- onmousemove** – Pointer moved while over the element.
- oncontextmenu** – The context (right-click) menu was requested.
- onwheel** – The mouse wheel was rotated.

Keyboard events:

- onkeydown** – A key was pressed down.
- onkeyup** – A key was released.
- onkeypress** – (Deprecated) A character key was pressed.

Form / input events:

- oninput** – The value of an input/textarea changed (fires per keystroke).
- onchange** – The value changed AND the element lost focus / was committed.
- onsubmit** – A <form> is being submitted.
- onreset** – A <form> is being reset.
- onfocus** – The element gained focus.
- onblur** – The element lost focus.
- onselect** – Text inside an input/textarea was selected.
- oninvalid** – A form control failed validation.

Clipboard events:

- oncopy** – Content was copied.
- oncut** – Content was cut.
- onpaste** – Content was pasted.

Drag-and-drop events:

- ondragstart** – Dragging started on a draggable element.
- ondrag** – The element is being dragged.
- ondragenter** – A dragged item entered a valid drop target.
- ondragover** – A dragged item is over a valid drop target.
- ondragleave** – A dragged item left a valid drop target.
- ondrop** – A dragged item was dropped on a target.
- ondragend** – Dragging finished.

Window / document-level events (typically on <body>):

- onload** – The resource and its dependents finished loading.
- onunload** – The document is being unloaded.
- onresize** – The window was resized.
- onscroll** – The element/document was scrolled.
- onerror** – A resource failed to load.

Media events (on <audio>/<video>):

- onplay, onpause, onended, ontimeupdate, onvolumechange, onloadeddata, oncanplay, onseeking, onseeked, onwaiting, ondurationchange.**

Attribute Index

Every HTML attribute, the element(s) it belongs to, and what it does — the reverse view of the per-element listings above. Rows marked *Global Attributes* apply to all elements (detailed in the previous section); element names link to their entries. Includes a few attributes the per-element entries don't call out individually (e.g. `dirname`), and the deprecated `align`/`bgcolor`/`border`/`color`).

ATTRIBUTE	BELONGS TO	DESCRIPTION
<code>accept</code>	<code><input></code>	Specifies the types of files that the server accepts (only for <code>type="file"</code>)
<code>accept-charset</code>	<code><form></code>	Specifies the character encodings that are to be used for the form submission
<code>accesskey</code>	<i>Global Attributes</i>	Specifies a shortcut key to activate/focus an element
<code>action</code>	<code><form></code>	Specifies where to send the form-data when a form is submitted
<code>align</code>	<i>Not supported in HTML 5.</i>	Specifies the alignment according to surrounding elements. Use CSS instead
<code>alt</code>	<code><area></code> , <code></code> , <code><input></code>	Specifies an alternate text when the original element fails to display
<code>async</code>	<code><script></code>	Specifies that the script is executed asynchronously (only for external scripts)
<code>autocomplete</code>	<code><form></code> , <code><input></code>	Specifies whether the <code><form></code> or the <code><input></code> element should have autocomplete enabled
<code>autofocus</code>	<code><button></code> , <code><input></code> , <code><select></code> , <code><text area></code>	Specifies that the element should automatically get focus when the page loads
<code>autoplay</code>	<code><audio></code> , <code><video></code>	Specifies that the audio/video will start playing as soon as it is ready
<code>bgcolor</code>	<i>Not supported in HTML 5.</i>	Specifies the background color of an element. Use CSS instead
<code>border</code>	<i>Not supported in HTML 5.</i>	Specifies the width of the border of an element. Use CSS instead
<code>charset</code>	<code><meta></code> , <code><script></code>	Specifies the character encoding
<code>checked</code>	<code><input></code>	Specifies that an <code><input></code> element should be pre-selected when the page loads (for <code>type="checkbox"</code> or <code>type="radio"</code>)
<code>cite</code>	<code><blockquote></code> , <code></code> , <code><ins></code> , <code><q></code>	Specifies a URL which explains the quote/deleted/inserted text
<code>class</code>	<i>Global Attributes</i>	Specifies one or more class names for an element (refers to a class in a style sheet)
<code>color</code>	<i>Not supported in HTML 5.</i>	Specifies the text color of an element. Use CSS instead
<code>cols</code>	<code><textarea></code>	Specifies the visible width of a text area
<code>colspan</code>	<code><td></code> , <code><th></code>	Specifies the number of columns a table cell should span
<code>content</code>	<code><meta></code>	Gives the value associated with the <code>http-equiv</code> or <code>name</code> attribute
<code>contenteditable</code>	<i>Global Attributes</i>	Specifies whether the content of an element is editable or not
<code>controls</code>	<code><audio></code> , <code><video></code>	Specifies that audio/video controls should be displayed (such as a play/pause button etc.)
<code>coords</code>	<code><area></code>	Specifies the coordinates of the area
<code>data</code>	<code><object></code>	Specifies the URL of the resource to be used by the object
<code>data-*</code>	<i>Global Attributes</i>	Used to store custom data private to the page or application
<code>datetime</code>	<code></code> , <code><ins></code> , <code><time></code>	Specifies the date and time
<code>default</code>	<code><track></code>	Specifies that the track is to be enabled if the user's preferences do not indicate that another track would be more appropriate
<code>defer</code>	<code><script></code>	Specifies that the script is executed when the page has finished parsing (only for external scripts)
<code>dir</code>	<i>Global Attributes</i>	Specifies the text direction for the content in an element
<code>dirname</code>	<code><input></code> , <code><textarea></code>	Specifies that the text direction will be submitted

ATTRIBUTE	BELONGS TO	DESCRIPTION
disabled	<button>, <fieldset>, <input>, <optgroup>, <option>, <select>, <textarea>	Specifies that the specified element/group of elements should be disabled
download	<a>, <area>	Specifies that the target will be downloaded when a user clicks on the hyperlink
draggable	<i>Global Attributes</i>	Specifies whether an element is draggable or not
enctype	<form>	Specifies how the form-data should be encoded when submitting it to the server (only for method="post")
enterkeyhint	<i>Global Attributes</i>	Specifies the text of the enter-key on a virtual keyboard
for	<label>, <output>	Specifies which form element(s) a label/calculation is bound to
form	<button>, <fieldset>, <input>, <label>, <meter>, <object>, <output>, <select>, <textarea>	Specifies the name of the form the element belongs to
formaction	<button>, <input>	Specifies where to send the form-data when a form is submitted. Only for type="submit"
headers	<td>, <th>	Specifies one or more headers cells a cell is related to
height	<canvas>, <embed>, <iframe>, , <input>, <object>, <video>	Specifies the height of the element
hidden	<i>Global Attributes</i>	Specifies that an element is not yet, or is no longer, relevant
high	<meter>	Specifies the range that is considered to be a high value
href	<a>, <area>, <base>, <link>	Specifies the URL of the page the link goes to
hreflang	<a>, <area>, <link>	Specifies the language of the linked document
http-equiv	<meta>	Provides an HTTP header for the information/value of the content attribute
id	<i>Global Attributes</i>	Specifies a unique id for an element
inert	<i>Global Attributes</i>	Specifies that the browser should ignore this section
inputmode	<i>Global Attributes</i>	Specifies the mode of a virtual keyboard
ismap		Specifies an image as a server-side image map
kind	<track>	Specifies the kind of text track
label	<track>, <option>, <optgroup>	Specifies the title of the text track
lang	<i>Global Attributes</i>	Specifies the language of the element's content
list	<input>	Refers to a <code><datalist></code> element that contains pre-defined options for an <code><input></code> element
loop	<audio>, <video>	Specifies that the audio/video will start over again, every time it is finished
low	<meter>	Specifies the range that is considered to be a low value
max	<input>, <meter>, <progress>	Specifies the maximum value
maxLength	<input>, <textarea>	Specifies the maximum number of characters allowed in an element
media	<a>, <area>, <link>, <source>, <style>	Specifies what media/device the linked document is optimized for
method	<form>	Specifies the HTTP method to use when sending form-data
min	<input>, <meter>	Specifies a minimum value
multiple	<input>, <select>	Specifies that a user can enter more than one value
muted	<video>, <audio>	Specifies that the audio output of the video should be muted
name	<button>, <fieldset>, <form>, <iframe>, <input>, <map>, <meta>, <object>, <output>, <param>, <select>, <textarea>	Specifies the name of the element
novalidate	<form>	Specifies that the form should not be validated when submitted
onabort	<audio>, <embed>, , <object>, <video>	Script to be run on abort
onafterprint	<body>	Script to be run after the document is printed

ATTRIBUTE	BELONGS TO	DESCRIPTION
onbeforeprint	<body>	Script to be run before the document is printed
onbeforeunload	<body>	Script to be run when the document is about to be unloaded
onblur	<i>All visible elements.</i>	Script to be run when the element loses focus
oncanplay	<audio>, <embed>, <object>, <video>	Script to be run when a file is ready to start playing (when it has buffered enough to begin)
oncanplaythrough	<audio>, <video>	Script to be run when a file can be played all the way to the end without pausing for buffering
onchange	<i>All visible elements.</i>	Script to be run when the value of the element is changed
onclick	<i>All visible elements.</i>	Script to be run when the element is being clicked
oncontextmenu	<i>All visible elements.</i>	Script to be run when a context menu is triggered
oncopy	<i>All visible elements.</i>	Script to be run when the content of the element is being copied
oncuechange	<track>	Script to be run when the cue changes in a <track> element
oncut	<i>All visible elements.</i>	Script to be run when the content of the element is being cut
ondblclick	<i>All visible elements.</i>	Script to be run when the element is being double-clicked
ondrag	<i>All visible elements.</i>	Script to be run when the element is being dragged
ondragend	<i>All visible elements.</i>	Script to be run at the end of a drag operation
ondragenter	<i>All visible elements.</i>	Script to be run when an element has been dragged to a valid drop target
ondragleave	<i>All visible elements.</i>	Script to be run when an element leaves a valid drop target
ondragover	<i>All visible elements.</i>	Script to be run when an element is being dragged over a valid drop target
ondragstart	<i>All visible elements.</i>	Script to be run at the start of a drag operation
ondrop	<i>All visible elements.</i>	Script to be run when dragged element is being dropped
ondurationchange	<audio>, <video>	Script to be run when the length of the media changes
onemptied	<audio>, <video>	Script to be run when something bad happens and the file is suddenly unavailable (like unexpectedly disconnects)
onended	<audio>, <video>	Script to be run when the media has reach the end (a useful event for messages like "thanks for listening")
onerror	<audio>, <body>, <embed>, , <object>, <script>, <style>, <video>	Script to be run when an error occurs
onfocus	<i>All visible elements.</i>	Script to be run when the element gets focus
onhashchange	<body>	Script to be run when there has been changes to the anchor part of the a URL
oninput	<i>All visible elements.</i>	Script to be run when the element gets user input
oninvalid	<i>All visible elements.</i>	Script to be run when the element is invalid
onkeydown	<i>All visible elements.</i>	Script to be run when a user is pressing a key
onkeypress	<i>All visible elements.</i>	Script to be run when a user presses a key
onkeyup	<i>All visible elements.</i>	Script to be run when a user releases a key
onload	<body>, <iframe>, , <input>, <link>, <script>, <style>	Script to be run when the element is finished loading
onloadeddata	<audio>, <video>	Script to be run when media data is loaded
onloadedmetadata	<audio>, <video>	Script to be run when meta data (like dimensions and duration) are loaded
onloadstart	<audio>, <video>	Script to be run just as the file begins to load before anything is actually loaded
onmousedown	<i>All visible elements.</i>	Script to be run when a mouse button is pressed down on an element
onmousemove	<i>All visible elements.</i>	Script to be run as long as the mouse pointer is moving over an element
onmouseout	<i>All visible elements.</i>	Script to be run when a mouse pointer moves out of an element
onmouseover	<i>All visible elements.</i>	Script to be run when a mouse pointer moves over an element
onmouseup	<i>All visible elements.</i>	Script to be run when a mouse button is released over an element
onmousewheel	<i>All visible elements.</i>	Script to be run when a mouse wheel is being scrolled over an element

ATTRIBUTE	BELONGS TO	DESCRIPTION
onoffline	<body>	Script to be run when the browser starts to work offline
ononline	<body>	Script to be run when the browser starts to work online
onpagehide	<body>	Script to be run when a user navigates away from a page
onpageshow	<body>	Script to be run when a user navigates to a page
onpaste	<i>All visible elements.</i>	Script to be run when the user pastes some content in an element
onpause	<audio>, <video>	Script to be run when the media is paused either by the user or programmatically
onplay	<audio>, <video>	Script to be run when the media has started playing
onplaying	<audio>, <video>	Script to be run when the media has started playing
onpopstate	<body>	Script to be run when the window's history changes.
onprogress	<audio>, <video>	Script to be run when the browser is in the process of getting the media data
onratechange	<audio>, <video>	Script to be run each time the playback rate changes (like when a user switches to a slow motion or fast forward mode).
onreset	<form>	Script to be run when a reset button in a form is clicked.
onresize	<body>	Script to be run when the browser window is being resized.
onscroll	<i>All visible elements.</i>	Script to be run when an element's scrollbar is being scrolled
onsearch	<input>	Script to be run when the user writes something in a search field (for <input type="search">)
onseeked	<audio>, <video>	Script to be run when the seeking attribute is set to false indicating that seeking has ended
onseeking	<audio>, <video>	Script to be run when the seeking attribute is set to true indicating that seeking is active
onselect	<i>All visible elements.</i>	Script to be run when the element gets selected
onstalled	<audio>, <video>	Script to be run when the browser is unable to fetch the media data for whatever reason
onstorage	<body>	Script to be run when a Web Storage area is updated
onsubmit	<form>	Script to be run when a form is submitted
onsuspend	<audio>, <video>	Script to be run when fetching the media data is stopped before it is completely loaded for whatever reason
ontimeupdate	<audio>, <video>	Script to be run when the playing position has changed (like when the user fast forwards to a different point in the media)
ontoggle	<details>	Script to be run when the user opens or closes the <details> element
onunload	<body>	Script to be run when a page has unloaded (or the browser window has been closed)
onvolumechange	<audio>, <video>	Script to be run each time the volume of a video/audio has been changed
onwaiting	<audio>, <video>	Script to be run when the media has paused but is expected to resume (like when the media pauses to buffer more data)
onwheel	<i>All visible elements.</i>	Script to be run when the mouse wheel rolls up or down over an element
open	<details>	Specifies that the details should be visible (open) to the user
optimum	<meter>	Specifies what value is the optimal value for the gauge
pattern	<input>	Specifies a regular expression that an <input> element's value is checked against
placeholder	<input>, <textarea>	Specifies a short hint that describes the expected value of the element
popover	<i>Global Attributes</i>	Specifies a popover element
popovertarget	<button>, <input>	Specifies which popover element to invoked
popovertargetaction	<button>, <input>	Specifies what happens to the popover element when the button is clicked
poster	<video>	Specifies an image to be shown while the video is downloading, or until the user hits the play button

ATTRIBUTE	BELONGS TO	DESCRIPTION
preload	<audio>, <video>	Specifies if and how the author thinks the audio/video should be loaded when the page loads
readonly	<input>, <textarea>	Specifies that the element is read-only
rel	<a>, <area>, <form>, <link>	Specifies the relationship between the current document and the linked document
required	<input>, <select>, <textarea>	Specifies that the element must be filled out before submitting the form
reversed		Specifies that the list order should be descending (9,8,7...)
rows	<textarea>	Specifies the visible number of lines in a text area
rowspan	<td>, <th>	Specifies the number of rows a table cell should span
sandbox	<iframe>	Enables an extra set of restrictions for the content in an <iframe>
scope	<th>	Specifies whether a header cell is a header for a column, row, or group of columns or rows
selected	<option>	Specifies that an option should be pre-selected when the page loads
shape	<area>	Specifies the shape of the area
size	<input>, <select>	Specifies the width, in characters (for <input>) or specifies the number of visible options (for <select>)
sizes	, <link>, <source>	Specifies the size of the linked resource
span	<col>, <colgroup>	Specifies the number of columns to span
spellcheck	<i>Global Attributes</i>	Specifies whether the element is to have its spelling and grammar checked or not
src	<audio>, <embed>, <iframe>, , <input>, <script>, <source>, <track>, <video>	Specifies the URL of the media file
srcdoc	<iframe>	Specifies the HTML content of the page to show in the <iframe>
srclang	<track>	Specifies the language of the track text data (required if kind="subtitles")
srcset	, <source>	Specifies the URL of the image to use in different situations
start		Specifies the start value of an ordered list
step	<input>	Specifies the legal number intervals for an input field
style	<i>Global Attributes</i>	Specifies an inline CSS style for an element
tabindex	<i>Global Attributes</i>	Specifies the tabbing order of an element
target	<a>, <area>, <base>, <form>	Specifies the target for where to open the linked document or where to submit the form
title	<i>Global Attributes</i>	Specifies extra information about an element
translate	<i>Global Attributes</i>	Specifies whether the content of an element should be translated or not
type	<a>, <button>, <embed>, <input>, <link>, <menu>, <object>, <script>, <source>, <style>	Specifies the type of element
usemap	, <object>	Specifies an image as a client-side image map
value	<button>, <input>, , <option>, <meter>, <progress>, <param>	Specifies the value of the element
width	<canvas>, <embed>, <iframe>, , <input>, <object>, <video>	Specifies the width of the element
wrap	<textarea>	Specifies how the text in a text area is to be wrapped when submitted in a form

Obsolete / Deprecated Elements

Removed from HTML5 — kept here only so you recognize them in old code. Each notes its modern replacement.

<acronym> DEPRECATED

removed in HTML5. Use `<abbr>` instead.

Was used to mark an acronym. Browsers showed a tooltip from the title attribute. `<abbr>` now covers both acronyms and abbreviations.

```
<abbr title="North Atlantic Treaty Organization">NATO</abbr>
```

Do not use `<acronym>`. Replace it with `<abbr>` (documented next).

<big> DEPRECATED

removed in HTML5. Use CSS instead.

Was used to render text one size larger than surrounding text.

```
<span style="font-size: larger;">Bigger text</span>
```

Do not use `<big>`. Use CSS: font-size: larger; (or a specific size).

<center> DEPRECATED

removed in HTML5. Use CSS instead.

Was a block element that horizontally centered its contents.

```
<div style="text-align: center;">Centered inline content</div>
```

Do not use `<center>`. Use CSS: text-align: center; for inline content, or margin: 0 auto; with a set width for block elements.

**** DEPRECATED

removed in HTML5. Use CSS instead.

Was used to set the font face, size, and color of text.

```
<span style="font-family: Arial; color: blue; font-size: 18px;">Styled text</span>
```

Do not use `<font>`. Its old attributes (color, face, size) are replaced by the CSS properties color, font-family, and font-size.

<strike> DEPRECATED

removed in HTML5. Use `<del>` or `<s>` instead.

Was used to render strikethrough text.

```
<del>removed</del> <s>no longer accurate</s>
```

Do not use `<strike>`. Use `<del>` for an edit/deletion, `<s>` for outdated content, or CSS text-decoration: line-through; for purely visual strikethrough.

`<tt>` DEPRECATED

removed in HTML5. Use CSS instead.

Was used for "teletype" monospace text.

```
<code>monospace</code>  ←!— or —→ <span style="font-family: monospace;">text</span>
```

Do not use `<tt>`. Use a semantic element like `<code>`, `<kbd>`, `<samp>`, or `<var>` where appropriate, or CSS font-family: monospace; for plain monospace text.

`<frameset>` DEPRECATED

removed in HTML5. Do not use.

Replaced the `<body>` element in old "frames" pages. It divided the window into a grid of independent `<frame>` regions, each loading a separate HTML document. Modern layouts use CSS (flexbox/grid) and, for embedding, `<iframe>`.

```
<frameset cols="25%,75%">
  <frame src="menu.html" />
  <frame src="content.html" />
</frameset>
```

cols – Defined the column layout as a comma-separated list of widths.
`cols="25%,75%"` – Two columns at 25% and 75% width.
`cols="200,*"` – A fixed 200px column and one filling the rest ("*").

rows – Defined the row layout the same way as cols, but vertically.
`rows="100,*"` – A 100px top row and a remaining-space bottom row.

`<frame>` DEPRECATED

removed in HTML5. Do not use.

Defined a single region within a `<frameset>`, loading its own document.

```
<frame src="page.html" name="contentFrame" />
```

src – The URL of the document to load into the frame.
`src="sidebar.html"` – Loads sidebar.html in this frame.

name – A name used so links (target="...") could load pages into this frame.
`name="main"` – Allows target="main" links to load here.

noresize – Boolean. Prevented the user from dragging to resize the frame.
`noresize` – Lock the frame size.

scrolling – Whether scrollbars appeared.
scrolling="yes" – Always show scrollbars.
scrolling="no" – Never show scrollbars.
scrolling="auto" – Show only when needed.

frameborder – Whether a border was drawn (1 = yes, 0 = no).
frameborder="0" – No border.

marginwidth / marginheight – Inner margins in pixels.
marginwidth="10" – 10px left/right padding inside the frame.

<noframes> DEPRECATED

removed in HTML5. Do not use.

Provided fallback content for browsers that could not display frames. Obsolete because all browsers now support standard content.

```
<noframes>
  Your browser does not support frames. <a href="content.html">View content</a>.
</noframes>
```

No attributes. It simply held fallback HTML.

<dir> DEPRECATED

removed in HTML5. Use `<ul>` instead.

Was an early element for directory/file listings, rendered as a list.

```
<ul>
  <li>file1.txt</li>
  <li>file2.txt</li>
</ul>
```

Do not use `<dir>`. Replace it with `<ul>`.

<basefont> DEPRECATED

removed in HTML5. Use CSS instead.

Was placed in `<head>` to set a default font color, size, and face for the entire document.

```
<style>
  body { font-family: Arial, sans-serif; font-size: 16px; color: #222; }
</style>
```

Do not use <basefont>. Set document-wide typography with CSS on the body (or :root) – font-family, font-size, and color – as shown above.

<applet> DEPRECATED

removed in HTML5. Use `<embed>` or `<object>`.

Was used to embed a Java applet. Java applets are obsolete and unsupported by modern browsers. Embed media/plugins with `<object>`, `<embed>`, `<iframe>`, `<video>`, or `<img>` as appropriate.

```
<object data="content.pdf" type="application/pdf" width="600" height="400">  
  Fallback content if the resource cannot be displayed.  
</object>
```

Do not use `<applet>`. Replace it with `<object>` or `<embed>` (documented below), or a purpose-built element such as `<iframe>` / `<video>` depending on the content.